A Project Report

on

IPL MATCH SCORE AND WIN PREDICTION SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology
in
Computer Science & Engineering
(Artificial Intelligence and Machine Learning)

Submitted By
Rinku – 2822306
Rahul – 2822450
Ankit - 2822307

Under the Supervision of
Ms. Neha Sharma
(Assistant Professor)
CSE (AI&ML)

Panipat Institute of Engineering & Technology, Samalkha, Panipat

Affiliated to

Kurukshetra University, Kurukshetra

(2022-2026)

DECLARATION

We certify that

i. The work presented in this project report is an authentic record of our own work under the guidance of our supervisor. It has not been submitted to any other Institute for the award of any other degree or diploma.
ii. Whenever we have used information (text, data, figure, photograph, chart, analysis, inference, etc.) from other sources, we have given due credit by citing it in the text of the report and providing its details in the references.
iii. We have followed the guidelines provided by the department for preparing the report.

_____

Rinku (2822306)

_____

Rahul (2822450)

_____

Ankit (2822307)

Project report title: IPL MATCH SCORE AND WIN PREDICTION SYSTEM Semester: 8th Semester

Date: April 24,2026

APPROVAL FROM SUPERVISOR

This is to certify that the project report entitled "*IPL MATCH SCORE AND WIN PREDICTION SYSTEM*" presented by "*Rinku (2822306), Rahul (2822450), Ankit (2822307)*" under my supervision is an authentic work. To the best of my knowledge, the content of this report has not been submitted for the award of any previous degree to anyone else.

It is recommended that the report be accepted as fulfilling this part of the requirements for the award of the degree.

_____

Name: Ms. Neha Sharma

Designation: Assistant Professor

Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning)

Date:24 April , 2026

_____

(Counter Signed by)

Dr. Devendra Prasad

Prof. & Head,

Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning)

Panipat Institute of Engineering & Technology
70 K.M., G.T. Karnal Road, NH-I, PATTIKALYANA - 132 102 Samalkha (Panipat) Haryana
Phone : 0180-3059200, 2569700, 2569599, 2569699, 2569799
Fax : 0180-2569800 Mobile : 09354613020
e-mail : info@piet.co.in web-site : www.piet.co.in

CERTIFICATE

This is to certify that the work embodied in this report, entitled "*IPL MATCH SCORE AND WIN PREDICTION SYSTEM*" carried out by "*Rinku (2822306), Rahul (2822450), Ankit (2822307)*" is approved for the degree of "*Bachelor of Technology (B.Tech.) in CSE (AI&ML)*" at the department of "*Artificial Intelligence and Machine Learning*",
Panipat Institute of Engineering and Technology, Samalkha.

_____

Internal Examiner

_____

External Examiner

Date:24 April, 2026
Place: Panipat

ACKNOWLEDGEMENTS

It gives us a great sense of pleasure to present the report of the Project undertaken during B. Tech. CSE (AI&ML), Final Year.

We would like to express our deepest gratitude to Ms. Neha Sharma, Assistant Professor, Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning), PIET, Samalkha for her exceptional dedication and major contributions that have played a pivotal role in the realization of this project. Her sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only her cognizant efforts that our endeavors have seen light of the day.

We would like to extend our sincere thanks to Dr. Devendra Prasad, Head Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning), PIET, Samalkha for his full support and assistance during the development of the project.

We are thankful to all faculty members of the department for their kind assistance, guidance, and cooperation during the development of our project.

Lastly, we would like to acknowledge our friends for their contribution in the completion of the project.

Rinku (2822306)

Rahul (2822450)

Ankit (2822307)

Date: April 24,2026

ABSTRACT

The growing availability of structured sports data and advancements in machine learning have significantly enhanced the field of cricket analytics. This project presents the development of an IPL analytics system designed to estimate two key live outcomes during a match: the projected final innings total and the batting team's win probability. The system is built using ball-by-ball data from 1,189 IPL matches sourced from Cricsheet, along with an engineered live-state dataset containing 282,552 observations spanning seasons from 2007 to 2026.

Unlike traditional approaches that rely only on basic match statistics such as runs, wickets, and overs, the proposed system incorporates a rich set of contextual features to improve prediction accuracy. These include momentum-based rolling windows, venue-specific performance, recent team form, team-versus-venue statistics, head-to-head records, player-level form, batter-versus-bowler history, and optional weather-based factors such as dew conditions. This combination enables the model to capture both immediate match dynamics and historical trends.

The backend of the system focuses on efficient data processing, feature engineering, and model inference. Multiple experiments were conducted using different training scopes and machine learning models under chronological evaluation to ensure realistic performance. The best-performing configuration utilizes an all-active-history training approach, a PyTorch-based entity embedding model for score prediction, and a calibrated CatBoost classifier for win probability estimation. These models effectively capture complex relationships between players, teams, and match conditions.

In the final evaluation, the score prediction model achieved a Mean Absolute Error (MAE) of 15.36 and a Root Mean Squared Error (RMSE) of 19.81. The win probability model achieved a log loss of 0.4922, a Brier score of 0.1681, and an accuracy of 67.1%, indicating reliable probabilistic predictions.

Overall, this project demonstrates a scalable and intelligent sports analytics system that combines advanced feature engineering with machine learning techniques to deliver accurate live match predictions. The platform can be further extended with additional data sources and advanced models, making it a practical tool for modern cricket analytics and decision-making.

TABLE OF CONTENTS

DECLARATION i

APPROVAL FROM SUPERVISOR ii

CERTIFICATE iii

ACKNOWLEDGEMENTS iv

ABSTRACT v

TABLE OF CONTENTS vi

LIST OF TABLES vii

LIST OF FIGURES viii

LIST OF ABBREVIATIONS ix

CHAPTER 1 - INTRODUCTION 1

CHAPTER 2 - LITERATURE REVIEW 5

CHAPTER 3 - PROBLEM OBJECTIVE 10

CHAPTER 4 - METHODOLOGY OF THE PROJECT 13

CHAPTER 5 - RESULTS AND ANALYSIS 33

CHAPTER 6 - SYSTEM IMPLEMENTATION AND TESTING 48

CHAPTER 7 - CONCLUSION AND FUTURE SCOPE 62

APPENDIX 1 - FLOWCHART AND ALGORITHM NOTES 65

APPENDIX 2 - SELECTED FEATURE NOTES 66

APPENDIX 3 - SHORT SUPPORTING CASE SNAPSHOTS 67

REFERENCES 68

LIST OF TABLES

Table 2.1: Approach Families Reviewed While Designing the Project 7

Table 4.1: Core Dataset Snapshot Used in the Final Project 14

Table 4.2: Live Feature Families and Their Project Value 17

Table 5.1: Scope Comparison That Guided Final Training Direction 33

Table 5.2: Live Score Model Comparison on the Key Evaluation Scope 34

Table 5.3: Live Win Model Comparison on the Key Evaluation Scope 36

Table 5.4: Pre-Match Model Performance Summary 41

Table 6.1: User-Facing Surfaces Built Around the Shared Prediction Core 48

LIST OF FIGURES

Figure 4.1: End-to-End IPL Prediction Workflow 13

Figure 5.1: Comparative View of Candidate Live Models 35

Figure 5.2: Held-Out Score Error Distribution 37

Figure 5.3: Held-Out Win Probability Calibration 37

Figure 6.1: Flask Live Prediction Interface 50

LIST OF ABBREVIATIONS

IPL: Indian Premier League

ML: Machine Learning

MAE: Mean Absolute Error

RMSE: Root Mean Squared Error

API: Application Programming Interface

CLI: Command Line Interface

GPU: Graphics Processing Unit

DFD: Data Flow Diagram

CHAPTER 1 – INTRODUCTION

1.1 TOPIC OF THE SYSTEM

The topic of this project is the design, development, and deployment of an IPL Match Score and Win Prediction System. The system applies supervised machine learning techniques to ball-by-ball Indian Premier League (IPL) match data in order to generate two real-time probabilistic predictions: an estimate of the likely final innings total for the batting side, and the batting side's probability of winning the match. These predictions are produced from any valid live match state during an ongoing IPL innings, meaning the system can be queried at any point from the first delivery of the match through to the final ball.

The topic is situated at the intersection of sports analytics, time-series feature engineering, and applied machine learning. It is relevant to the academic field of Computer Science and Artificial Intelligence because it demands solutions to several non-trivial engineering challenges: how to reconstruct a live match state from ball-by-ball records, how to compute historical support features without introducing temporal data leakage, how to evaluate probabilistic outputs honestly, and how to package a trained model inside a multi-surface software system that is usable by real end users.

The system addresses a practical gap in publicly available cricket analytics tools. While commercial platforms provide live scoring and some statistical overlays, few publicly documented systems combine leakage-safe feature engineering, probability-calibrated win estimation, and multiple deployable interfaces in a single open and reproducible project. This system attempts to fill that gap at an academic level, demonstrating what is achievable using publicly available ball-by-ball data sources and open-source machine learning libraries within a final-year undergraduate engineering project.

1.2 OVERVIEW OF THE PROJECT

The IPL Match Score and Win Prediction System is an end-to-end machine learning project that begins with raw ball-by-ball cricket data files and concludes with a deployed, multi-surface prediction service. The project covers every stage of the machine learning lifecycle: data acquisition and normalization, feature engineering and support table construction, model training and comparison, probability calibration, interface development, automated testing, and documentation.

The source data consists of 1,189 raw IPL match files in Cricsheet CSV2 format, covering IPL seasons from 2007/08 through the first portion of the 2026 season. After preprocessing, the primary training dataset contains 282,552 rows and 62 engineered columns, with each row representing a single legal-ball match state during an IPL innings. The breadth and depth of this dataset — nearly three hundred thousand training examples spanning nearly two decades of IPL cricket — allow the models to learn stable patterns across different venues, team compositions, and scoring environments.

Two separate prediction tasks are addressed. The first is a regression task: given the current match state, predict the final innings total. The second is a probabilistic classification task: given the same state, estimate the batting team's win probability. Both tasks are evaluated under strict chronological train-validation-test splitting to prevent temporal leakage, with metrics appropriate to each output type: MAE and RMSE for regression, log loss and Brier score for probability estimation.

Four model families were compared across two training scopes: HistGradientBoosting (CPU baseline), XGBoost (GPU-accelerated gradient boosting), CatBoost (categorical-aware gradient boosting), and a PyTorch entity-embedding model for tabular data. The strongest combination, a Torch entity-embedding regressor for score prediction paired with calibrated CatBoost GPU for win probability, achieved a held-out MAE of 15.36 runs, RMSE of 19.81 runs, and log loss of 0.4922 under the all-active-history training scope.

The trained models are served through four interfaces: a Flask web application, a Streamlit analytics dashboard, a JSON REST API endpoint, and a command-line interface. All four interfaces call the same shared inference module, guaranteeing consistent predictions regardless of access method. An automated pytest suite verifies the correctness of the inference logic, alias normalization, overs parsing, and web route behavior.

1.3 PROJECT BACKGROUND

T20 cricket, the format on which the Indian Premier League is based, is one of the most unpredictable and fast-changing formats of the sport. A match can shift direction dramatically within a single over. A batting side that appears to be on course for a comfortable total can lose several wickets in quick succession and find its final score severely curtailed. Conversely, a chasing side that looks far behind the required rate can launch a devastating assault in the death overs and win with balls to spare. This inherent volatility is central to the appeal of T20 cricket and simultaneously makes it a challenging and interesting prediction domain.

The Indian Premier League adds further complexity to the prediction problem. The IPL is a franchise-based Twenty20 competition featuring the best professional cricketers from around the world playing alongside Indian domestic talent. Ten franchises compete across eight to ten different venues each season, with varying pitch and outfield conditions, different stadium dimensions, and local weather patterns that can influence scoring rates. Player line-ups change match to match based on team selection, injury, and form. Team strategies evolve across seasons, and the overall scoring environment has drifted upward significantly: the average first-innings total in 2024 was approximately 190 runs, compared to approximately 161 runs in the earliest seasons of the league. These shifts make simple historical averages unreliable as prediction inputs without temporal correction.

Traditional approaches to cricket prediction fall into two broad categories. Rule-based heuristics — such as the Duckworth-Lewis method — express match state in terms of resources remaining (wickets and overs) and use fixed mathematical functions to project likely totals or adjust targets. These approaches are fast, interpretable, and institutionally validated, but they do

not account for venue-specific scoring norms, team form, player quality matchups, or recent momentum. The second category is machine learning classifiers trained on end-of-match summary data, which attempt to predict match winners from pre-match inputs. These models bypass the live prediction problem entirely and cannot answer questions about a match that is currently in progress.

The present project was motivated by the desire to go beyond both of these established approaches. By building a ball-by-ball feature dataset that captures the live innings state, venue context, team quality, player form, and short-term momentum, the system can make meaningful predictions at any point during a match rather than only before it starts or after it ends. This represents a substantially more informative prediction capability, and one that is of direct practical interest to cricket fans, commentators, and analysts who want a data-grounded view of how a live match is likely to evolve.

The choice to use the IPL as the exclusive domain was motivated by data availability and scope manageability. Cricsheet provides comprehensive ball-by-ball IPL records in a well-documented open format. The IPL's structured franchise system provides consistent team identities across seasons, making it easier to build reliable historical support features. The two-decade IPL archive provides enough data to train models with meaningful statistical power. And the IPL's broad following in India makes a working prediction system relevant to a large potential user community.

1.4 SCOPE OF THE PROJECT

The scope of this project is explicitly bounded to the Indian Premier League in the Twenty20 format. The system does not cover other cricket formats (One Day Internationals, Test matches, or other T20 variants) and does not generalize to other T20 leagues such as the Big Bash League or the Caribbean Premier League. This deliberate restriction allows the project to build IPL-specific normalization rules, support tables, active-team filters, and venue statistics without needing to handle the different player pools and competition structures of other leagues.

Within the IPL, the project covers all seasons from 2007/08 through the beginning of the 2026 season, using ball-by-ball data available through the Cricsheet open-data archive. The dataset is filtered for live-state modeling to include only rows from matches involving the ten currently active IPL franchises: Mumbai Indians, Chennai Super Kings, Royal Challengers Bengaluru, Kolkata Knight Riders, Delhi Capitals, Punjab Kings, Rajasthan Royals, Sunrisers Hyderabad, Gujarat Titans, and Lucknow Super Giants. Historical data from now-defunct franchises is used in preprocessing to compute full support tables but is excluded from the final model training scope through an active-team filter.

The prediction scope covers both first and second innings of standard twenty-over matches. The system handles reduced-over matches in a limited fashion but does not guarantee correct predictions for weather-interrupted matches where the Duckworth-Lewis-Stern method would apply. The prediction outputs include a projected final innings total, a batting-side win probability, a projected scoring range, and a runs-versus-par context indicator. The system does

not provide tactical recommendations, optimal bowling changes, optimal batting strategies, or player ranking outputs.

The software scope includes a Flask web application, a Streamlit analytics dashboard, a JSON REST API endpoint, and a command-line interface — all backed by the same shared inference module. The deployment scope is local-area-network demonstration, not public production deployment. The system is not connected to any live cricket scoring API and requires manual data entry for current match state at the time of submission. External commercial data sources, betting market feeds, and proprietary player-tracking data are all outside scope.

These boundaries are appropriate for a final-year B.Tech. project. They are tight enough to be achievable within the project timeline and broad enough to demonstrate the full machine learning system lifecycle from data ingestion through deployed interfaces. The core methodology — build a leakage-safe ball-by-ball feature dataset, compare model families honestly, calibrate probability outputs, and package predictions into testable interfaces — is generic enough to inspire similar projects in other sports and other data domains.

1.5 GOALS AND OBJECTIVES

The project was designed around a well-defined set of goals that guided every engineering and design decision. These goals span data quality, model performance, software delivery, and academic reporting. Meeting all of them together is what distinguishes this project from a simple notebook experiment.

The primary technical goals are listed below:

- Acquire and normalize 1,189 raw IPL match files from Cricsheet CSV2 format, resolving all known team name and venue name variants through a comprehensive alias normalization table to prevent identity fragmentation in the feature dataset.
- Reconstruct every legal ball state within each innings to produce a ball-by-ball feature dataset containing 282,552 rows and 62 columns, with complete metadata and all derived features computed without any temporal data leakage.
- Compute nine historical support tables (venue statistics, team form, team-venue form, matchup form, batter profiles, bowler profiles, batter-bowler matchup history, active-team list, and player pool) in strict chronological order so that every historical feature value uses only information from prior matches.
- Train and compare at least four model families for each prediction task under identical chronological train-validation-test splits and evaluation metrics, producing a documented model selection record.
- Select the best-performing model pair and persist it in a versioned model registry with full performance metadata, scope information, and artifact pointers.
- Apply isotonic-regression probability calibration to the win probability model and verify the improvement using log loss, Brier score, and five-bucket calibration-gap analysis.

- Build a shared inference module that constructs the live feature frame and invokes both models consistently, serving Flask, Streamlit, REST API, and CLI without any duplication of feature engineering logic.
- Implement an automated pytest suite covering alias normalization, feature frame consistency, overs parsing correctness, and web route behavior.
- Implement a monitoring layer that logs prediction events and compares predicted outcomes to resolved match results for drift detection.

The primary academic goals are:

- Produce a project report that traces every major result back to a specific design decision, making the reasoning behind performance improvements fully reproducible and defensible under faculty or viva examination.
- Demonstrate chronologically honest evaluation throughout, so that all reported performance figures reflect the model's expected behavior on genuinely unseen future seasons.
- Apply software engineering principles including modularity, separation of concerns, low coupling, automated testing, and artifact traceability throughout the implementation.

1.6 BENEFITS OF THE PROJECT

The IPL Match Score and Win Prediction System provides benefits across three dimensions: practical, academic, and methodological. Each dimension adds a different kind of value, and together they make the project more than a routine machine learning exercise.

Practically, the system provides cricket followers, commentators, and analysts with a richer, more quantitative view of live match dynamics. Instead of relying solely on the visible scoreboard or intuitive run-rate arithmetic, a user can consult the system for a projected final total, a calibrated win probability, a scoring range that reflects genuine uncertainty, and a comparison of the current scoring pace against the historical venue baseline. These outputs provide an evidence-based analytical frame that can enhance live commentary, fan engagement, and analytical discussion during IPL broadcasts.

From a cricket analytics perspective, the system demonstrates how significantly the predictive value of a model depends on the quality of the features it receives, not just the algorithm used. By separating the live prediction problem into score regression and win probability estimation, and by providing the models with venue context, team form, player profiles, momentum windows, and chase pressure features, the system consistently outperforms simpler heuristics. This reinforces the importance of domain-informed feature engineering in any sports analytics application.

Academically, the project provides a concrete, reproducible case study in how to build a production-quality machine learning pipeline as part of a final-year undergraduate project. It demonstrates best practices that are often taught in theory but rarely executed together: chronological train-test splitting, support table construction outside the training loop, probability calibration, modular interface design, automated testing, and versioned model registration.

Future students can use this project as a reference implementation for applied ML work in sports and other domains.

Methodologically, the project demonstrates that the largest performance gains in a structured tabular ML problem come from data engineering decisions rather than algorithm selection. The improvement from switching training scopes (recent-only to all-active-history) exceeded the improvement from switching model families. This lesson — that data quality and scope design matter more than architecture novelty — is one of the most practically valuable findings of the project and is directly applicable to industrial ML projects across any domain.

A further benefit is the project's extensibility. The support-table architecture, the shared inference contract, the monitoring module, and the multi-surface deployment structure were all designed with future development in mind. Adding a new interface, incorporating a new season's data, or experimenting with a new model family each requires changes only to well-defined, isolated components rather than modifications to an entangled monolithic codebase.

1.7 RELEVANCE OF THE PROJECT

The relevance of the IPL Match Score and Win Prediction System extends across technological, educational, and social dimensions. Understanding these dimensions helps explain why the project was designed in the way it was and why the methodology choices made throughout the project matter beyond the specific domain of cricket prediction.

Technologically, the project demonstrates mastery of several tools and techniques that are central to modern applied machine learning: gradient boosting frameworks (XGBoost, CatBoost), entity-embedding neural networks for tabular data (PyTorch), probability calibration (scikit-learn's CalibratedClassifierCV), web application development (Flask, Streamlit), REST API design, automated testing (pytest), and reproducible data pipeline construction (pandas, numpy). Each of these technologies is widely used in industry-level data science and machine learning engineering roles, making the project directly relevant to the employment landscape for a Computer Science and AI graduate.

Educationally, the project addresses a common weakness in academic machine learning courses: the gap between learning individual techniques in isolation and integrating them into a coherent, production-ready system. Most academic projects demonstrate one technique on one dataset and produce results in a Jupyter notebook. This project demonstrates the full pipeline from raw data to deployed interfaces, including components — alias normalization, chronological splitting, probability calibration, model registry management, monitoring — that are rarely covered in coursework but are essential in real-world deployment. The project therefore provides educational value that extends well beyond the specific topic of cricket prediction.

Socially, cricket is deeply embedded in Indian culture, and the IPL is one of the most followed sports leagues in the world. A system that can express live match dynamics in probabilistic, quantitative terms has the potential to elevate the quality of fan engagement and sports journalism by moving beyond pure intuition. At the same time, the system was designed

with responsible use in mind: it presents predictions with explicit uncertainty information, does not support gambling applications, and does not claim to eliminate the genuine randomness that makes cricket exciting. This responsible design approach is itself educationally relevant, demonstrating that AI systems should be built with awareness of how their outputs will be interpreted and used.

1.8 CHALLENGES AND CONSTRAINTS

Several significant challenges had to be overcome during the development of this project. Understanding these challenges is important because they explain many of the design choices described in later chapters — choices that might otherwise seem unnecessarily complex.

The most fundamental challenge was temporal data leakage. When historical features such as venue scoring averages, team form scores, and player career statistics are computed using the full dataset archive, they inadvertently embed information from future matches into features used for training rows from earlier matches. A model trained on leaked features appears to perform well in evaluation but fails to maintain that performance in real deployment because it was implicitly trained on information it would not have access to in live use. Eliminating leakage required recomputing all support table features in strict chronological order, match by match, which added substantial preprocessing complexity and computation time but was non-negotiable for the integrity of the results.

The second major challenge was data heterogeneity across IPL seasons. Team names, venue names, and season labels appear in multiple textual variants across the 1,189 Cricsheet match files. The franchise now known as Royal Challengers Bengaluru has appeared as Royal Challengers Bangalore in many files. Delhi Capitals was previously called Delhi Daredevils. Several venue names appear in truncated or alternate forms. Without comprehensive alias normalization, these variants would create artificial category fragmentation that weakens statistical estimates and causes join failures when merging support tables with the main feature dataset. Building and maintaining the alias table was a time-consuming but essential preprocessing step.

The third challenge was training scope selection. A narrow recent-only training scope is temporally close to the deployment period but may contain too few rows to learn reliable venue, team, and player statistics once chronological splits are applied. A broad all-history scope provides more data but risks including obsolete patterns from defunct franchises and much earlier scoring environments. Finding the right balance — the all-active-history scope that includes full historical depth while restricting to currently active franchises — required empirical comparison of both approaches and careful analysis of the scope-performance trade-off.

The fourth challenge was probability calibration for the win model. A classification model that achieves strong accuracy may still produce poorly calibrated probability outputs that overstate or understate confidence systematically. For a system that displays win percentages to users, calibration matters more than accuracy in many respects. Applying and validating

isotonic regression calibration, choosing the right calibration evaluation metrics, and verifying that calibration held across different probability bands all required careful experimental work.

Additional constraints included limited access to GPU hardware for intensive training experiments, reliance exclusively on publicly available data without commercial cricket data feeds, and the requirement to complete the project within a student timeline while maintaining academic report quality. Each constraint shaped specific aspects of the final design, from the choice of model families evaluated to the scope of the deployment surfaces implemented.

1.9 STRUCTURE OF THE REPORT

The remainder of this report is organized into six further chapters, three appendices, and a references section. Each chapter builds on the previous ones, progressing logically from the review of relevant prior work through the technical methodology, experimental results, system implementation, and final conclusions.

Chapter 2 presents a systematic review of the literature relevant to cricket analytics, structured-data machine learning, and sports prediction systems. It identifies the key methodological gaps in prior work that directly motivated the design choices adopted in this project, and explains how this project addresses each identified gap.

Chapter 3 defines the problem formally, lists the specific research questions and technical objectives, describes the scope boundaries in detail, and enumerates the complete set of project deliverables with a status review.

Chapter 4 describes the complete methodology including system architecture, end-to-end workflow design, dataset construction and characterization, data preprocessing operations, feature engineering rationale, model training strategy, exploratory data analysis, best-model selection criteria, and user interface design philosophy.

Chapter 5 presents the experimental results, organized around the key comparison axes of training scope and model family. It includes model performance tables, error analysis by wicket state and innings phase, calibration-bucket analysis, scenario-level interpretation, and a discussion of the factors that produced the strongest recorded result.

Chapter 6 describes the system implementation in detail, covering the shared inference architecture, the Flask and Streamlit interfaces, the REST API and CLI, software engineering principles applied, the dependency stack, automated testing, artifact traceability, workspace organization, and the monitoring module.

Chapter 7 concludes the report with a summary of achievements, key lessons learned, current limitations, and directions for future work including specific technical extensions that build naturally on the existing infrastructure.

Appendix 1 describes the prediction algorithm as an ordered step-by-step procedure. Appendix 2 notes the analytical significance of key engineered features. Appendix 3 records sample prediction outputs across representative match scenarios.

1.10 CONCLUSION

This introductory chapter has established the topic, context, motivation, scope, goals, benefits, relevance, challenges, and structure of the IPL Match Score and Win Prediction System project. The project addresses a practical and technically demanding problem — providing real-time, probabilistically calibrated predictions at any point during live IPL matches — through a disciplined combination of data engineering, comparative model evaluation, probability calibration, and multi-surface software deployment.

The project is relevant to both the academic evaluation context and the broader fields of applied machine learning and sports analytics. It demonstrates that the most important contributions in an applied ML project are often the least visible: leakage control, scope design, calibration quality, and software engineering discipline. These contributions produced more performance improvement than the choice of algorithm, and this lesson is documented carefully throughout the report.

The most significant design insight of the project is that strong IPL prediction depends on representing the live match state richly and honestly, not on choosing the most sophisticated algorithm for a poorly represented state. This principle, established in this introduction and validated empirically in the results chapter, runs as a consistent thread through every chapter of the report that follows.

CHAPTER 2 – LITERATURE REVIEW

2.1 INTRODUCTION

This chapter reviews the literature that is most relevant to the design and evaluation of the IPL Match Score and Win Prediction System. The review draws from three thematic streams: classical cricket analytics and heuristic prediction methods, machine learning methodology for structured tabular data, and applied sports prediction project reports. Together, these streams provide the intellectual context in which the present project sits and identify the specific gaps that motivated the design decisions documented in later chapters.

The purpose of a literature review in an applied engineering project is not merely to demonstrate familiarity with prior work. It is to justify design choices by connecting them to prior evidence and identified gaps. Every significant methodological decision in this project — the choice to build a ball-by-ball live-state dataset, the emphasis on chronological evaluation, the decision to calibrate the win model, the choice to separate score and win tasks into different model families — is traceable to insights or identified deficiencies in the literature reviewed in this chapter.

One important finding that emerged early in this review is that the quality of the live match-state representation matters more than the choice of model algorithm for cricket prediction tasks. Systems that use impoverished state representations — whether due to data limitations, feature design weaknesses, or evaluation leakage — consistently produce unreliable results regardless of the model sophistication applied. This finding shaped the project's emphasis on feature engineering and leakage control over algorithm novelty.

The review is organized as follows. Section 2.2 reviews key literature across the five relevant technical themes. Section 2.3 synthesizes the main findings from the review. Section 2.4 explains how the identified gaps were addressed in the present project. Section 2.5 explains why this review was important to the final project design rather than serving as a formality.

2.2 REVIEW OF KEY LITERATURE

The literature reviewed in this project spans five thematic areas: classical cricket score heuristics, gradient boosting for structured prediction, categorical-aware and entity-embedding models, probability calibration methodology, and applied sports analytics workflows. Each area contributed specific insights and identified specific weaknesses that influenced the project design.

Classical cricket score heuristics: Early cricket analytics work established current run rate, wickets in hand, and overs remaining as the primary predictors of match outcome. The Duckworth-Lewis method (Duckworth and Lewis, 1998) formalized the relationship between remaining batting resources and expected score for weather-interrupted One Day Internationals. This resource-based approach is elegant and institutionally validated, but it was designed for resource-allocation in interrupted matches rather than for live continuous prediction. Applied to T20 prediction, the method cannot distinguish between a score of 80/2 in a high-scoring venue

and the same score in a low-scoring venue, cannot account for team quality differences, and cannot capture the momentum trajectory of recent overs. The gap identified from this stream is that heuristic formulas, while useful as baselines, cannot represent the multivariate contextual information that determines likely match outcomes in T20 cricket.

Sankaranarayanan, Sattar, and Lakshmanan (2014) applied data mining techniques to ODI cricket simulation and prediction. Their work demonstrated that machine learning could outperform simple heuristics on cricket prediction tasks, but it operated primarily on match-summary features rather than ball-by-ball live states. This approach produces models that can explain what happened after a match is complete but cannot make meaningful predictions while the match is still in progress. The gap from this work is the absence of a true live-state prediction framework that can answer queries at any intermediate ball within a match.

Gradient boosting for structured prediction: The introduction of XGBoost (Chen and Guestrin, 2016) established that regularized boosted trees could be trained efficiently on large structured datasets with strong predictive quality. XGBoost demonstrated consistent advantages over earlier ensemble methods on tabular data competitions, and its GPU-accelerated training backend made it practical for datasets of the scale encountered in the present project. The gradient boosting literature consistently showed that tree-based ensemble methods outperform linear models and many neural network architectures on mixed categorical-numeric tabular data, justifying the choice of gradient boosting as the primary baseline family in this project.

CatBoost (Prokhorenkova et al., 2018) extended this line by introducing ordered, leakage-safe handling of categorical features. Rather than encoding categorical variables as one-hot vectors or using naive target encoding that leaks label information, CatBoost computes ordered target statistics that simulate what the model would have seen at each training example if it had only observed prior data. This ordered approach is especially relevant for cricket analytics because teams, venues, phases, and players are all high-cardinality categorical entities that carry strong predictive signal. The gap left by this line of work is that even excellent categorical handling does not remove the need for careful chronological construction of historical features outside the model — CatBoost's internal ordering only protects against within-batch leakage, not across-season temporal leakage.

Entity embeddings for tabular categorical data: Guo and Berkhahn (2016) demonstrated that learning dense low-dimensional embeddings for categorical variables in neural networks can outperform one-hot encoding on datasets where categorical identities carry rich relational information. Entity embeddings allow the model to place similar venues, teams, or players close together in embedding space, capturing similarity relationships that a one-hot representation cannot express. This approach is attractive for cricket analytics because franchises, venues, and players are not exchangeable categories — their identities carry meaningful signal about historical scoring patterns, playing styles, and form cycles. The present project evaluated a PyTorch entity-embedding model and found it achieved the strongest score prediction results once the all-active-history training scope provided sufficient data to stabilize the embeddings.

Probability calibration: Niculescu-Mizil and Caruana (2005) established through systematic empirical analysis that probability calibration is a distinct requirement from classification accuracy. A model may rank outcomes correctly while systematically over-stating or under-stating its confidence in specific probability ranges. For a live cricket win-probability system that displays percentages to users, calibration matters especially because users interpret the displayed number as a direct statement about likelihood — not as an ordinal ranking. Isotonic regression post-processing emerged from this line of work as an effective non-parametric calibration technique. The present project applied this directly to the CatBoost win classifier and verified the improvement using log loss, Brier score, and a five-bucket calibration-gap analysis.

Sports analytics workflow papers: A large body of applied project work in sports analytics — ranging from student projects to published papers — demonstrates pipelines for data collection, feature construction, and outcome prediction across various sports. These works are valuable because they show what full-stack sports prediction projects look like in practice. However, a consistent pattern in this literature is that projects stop at the experimental stage, reporting notebook-level metrics without integrating the trained model into a deployable, testable software system. A second consistent weakness is the use of random train-test splits rather than time-ordered chronological splits, which systematically inflates reported performance by mixing future seasons into the training data. The present project was explicitly designed to address both of these recurring weaknesses, treating deployment quality and evaluation integrity as first-class requirements alongside model performance.

Open data sources: Cricsheet (2024) makes modern ball-by-ball cricket analytics possible by providing freely accessible, well-documented event-level records and match metadata for professional cricket competitions. The ball-by-ball Cricsheet files are the primary data source for this project. Their value goes beyond simple data access: they enable full reproducibility because another researcher can rebuild the same match-state feature table from the same source files. However, raw data availability does not automatically produce a trustworthy training set. The present project still required normalization of team and venue names, conversion of overs notation into legal-ball counts, chronological reconstruction of support table features, and multiple layers of data quality validation before the raw files could support reliable modeling.

2.3 SUMMARY OF FINDINGS

Synthesizing the literature reviewed above reveals several consistent findings that shaped the design of the present project.

First, heuristic approaches based on run rate and resource tables provide useful baselines for cricket prediction but cannot represent the full contextual richness of a live T20 match state. Venue effects, team form, player matchups, and recent momentum all contribute independent predictive signal that heuristics cannot encode, motivating a machine learning approach with rich feature engineering.

Second, gradient boosting methods — specifically XGBoost and CatBoost — are the most reliably strong performers on structured tabular data with mixed categorical and numerical

features. These methods consistently outperform simpler linear models and require substantially less preprocessing than deep learning alternatives on typical tabular datasets. They therefore represent the correct starting point for model comparison on ball-by-ball cricket data.

Third, entity-embedding neural networks for tabular data can outperform gradient boosting when the dataset is large enough to support stable embedding learning and when categorical identities carry rich relational signal. In the present project, the all-active-history scope provided sufficient data for entity embeddings to demonstrate clear advantages on the score regression task.

Fourth, probability calibration is non-negotiable for win prediction systems that display probabilities to users. Classification accuracy is a misleading primary metric for this task because a model can achieve high accuracy by performing well on obvious states while remaining poorly calibrated in the ambiguous middle range where user trust matters most. Log loss and Brier score must be reported alongside accuracy, and post-hoc calibration (isotonic regression or Platt scaling) should be applied and verified.

Fifth, chronological evaluation is essential for any sports prediction system intended for real deployment. Random splits inflate reported performance by allowing future information to leak into the training data, systematically producing estimates that cannot be maintained in production. Only season-based chronological splits produce honest performance estimates for temporal prediction problems.

Sixth, most applied cricket prediction projects stop at the experimental stage. There is a persistent and important gap between notebook-level model evaluation and production-quality system deployment with consistent inference logic, automated testing, and versioned artifacts.

2.4 ADDRESSING IDENTIFIED GAPS

The present project was designed specifically to address each of the identified gaps in the prior literature. The response to each gap is described below, with reference to the specific design decisions made to close it.

The gap in feature richness — the reliance on run rate and wickets alone — is addressed by constructing a 62-column live-state feature set that combines basic scoreboard state, innings progress indicators, momentum windows, venue historical statistics, team form indices, team-venue interaction features, matchup form scores, and player-level batting and bowling profiles. This feature set is substantially richer than any feature representation found in prior academic cricket prediction work and directly improves predictive accuracy in the held-out evaluation.

The gap in evaluation integrity — the use of random train-test splits — is addressed by enforcing strict chronological ordering at every stage. The main training, validation, and test splits are defined by season boundaries. All historical support features are computed in match-by-match chronological order so that no feature value at any ball state uses information from future matches. This is the most consequential single design decision in the project: without it, the reported results would be overly optimistic and would not reflect real deployment performance.

The gap in probability calibration is addressed by applying isotonic regression calibration to the CatBoost win model after initial training, then verifying the improvement using log loss, Brier score, and a calibration-bucket analysis that compares predicted probability bands to empirical win rates in those bands. The project reports both uncalibrated and calibrated model performance, making the added value of calibration explicitly measurable.

The gap in deployment quality is addressed by implementing a shared inference module, four user-facing surfaces, an automated pytest suite, a model registry, and artifact traceability records. The project can be rebuilt from raw data to a working prediction service by running a documented sequence of commands, demonstrating genuine reproducibility. The shared inference contract ensures that improving the model logic in one place automatically improves all four interfaces, eliminating the risk of consistency drift.

2.5 IMPORTANCE OF THIS REVIEW TO THE PROJECT

The literature review in this project was not a formality conducted after the main design decisions had already been made. It was conducted during the early design phase and directly influenced the methodology described in Chapter 4. Several concrete design decisions trace back to specific findings in this review.

The decision to use gradient boosting as the primary model family baseline, and to compare against entity-embedding approaches, came directly from the literature's consistent evidence that tree-based methods outperform alternatives on mixed tabular data while entity embeddings offer potential advantages on high-cardinality categorical problems with sufficient data.

The decision to evaluate win models using log loss and Brier score rather than accuracy alone came from the calibration literature's systematic evidence that accuracy is an inadequate metric for probability-producing classifiers.

The decision to enforce chronological training and evaluation came from recognizing that the vast majority of prior cricket prediction papers had used random splits, which explained why their reported results so consistently failed to transfer to deployment settings.

The decision to build the full system through to deployment — rather than stopping at notebook metrics — came from observing how consistently prior work stopped at the experimental stage, and from the judgment that a final-year engineering project should demonstrate the full ML system lifecycle, not just the modeling component.

In retrospect, the literature review shaped more of the final project than any other single activity. It prevented several common methodological mistakes, justified the investment in feature engineering over algorithm experimentation, and established the standard of evidence against which the final results are reported. For all these reasons, this chapter occupies a central position in the report rather than being relegated to a brief background section.

CHAPTER 3 – PROBLEM OBJECTIVE

3.1 PROBLEM STATEMENT

The central problem addressed by this project is the prediction of Indian Premier League match outcomes from live ball-by-ball match states. Specifically, given a snapshot of an ongoing IPL innings that includes the current runs scored, wickets lost, overs bowled, venue, batting team, bowling team, innings number, and optionally the current striker's and bowler's names, the system must produce two outputs: a probabilistic estimate of the final innings total and the batting side's probability of winning the match.

This problem formulation is substantially stronger than either pre-match prediction or post-match classification. Pre-match prediction must work without any live match state, relying entirely on team identities, venue priors, and broad form indicators, which provides very limited predictive signal. Post-match classification learns from complete match outcomes and cannot answer questions about a match that is currently in progress. Live prediction from an intermediate state is the most practically useful and technically demanding formulation because it must answer the question 'given everything we know right now, what is likely to happen next?' at every ball of the match.

The problem is further complicated by the structural asymmetry between the two innings. In the first innings, the model must project how the current batting side will perform over the remaining overs without knowing what target will be required. In the second innings, the model must additionally account for the chase target, the required run rate, and the gap between the required rate and the current run rate. States that look identical on the basic scoreboard can mean very different things in the two innings contexts: 80/2 after ten overs is a comfortable position in the first innings and may be comfortable or dangerous in the second innings depending on the target.

Formally, the project defines two learning problems. The first is a regression problem: given the live feature vector X for a particular ball b in the innings, learn a function f such that f(X) approximates the final innings total T. The second is a calibrated classification problem: given a related feature vector X, learn a function g such that g(X) provides a well-calibrated estimate of P(batting team wins | X). Both functions must be constrained to use only information available at ball b — specifically, they must not use any data from balls after b in the same innings or from future matches relative to the current match date.

The specific challenge of maintaining this information constraint throughout the entire pipeline — from support table construction through model training and evaluation — is what makes this project technically distinctive. Many prior systems have implicitly violated this constraint by using features or evaluation protocols that embed future information, producing results that look strong in evaluation but fail in deployment. The present project treats information-constraint compliance as a first-class engineering requirement, not an afterthought.

3.2 OBJECTIVES OF THE PROJECT

The project objectives are organized into immediate technical goals and broader academic goals. The technical goals define what the system must accomplish and how its performance will be measured. The academic goals define the standards of documentation, explanation, and evaluation that the project must meet.

The technical objectives of the project are:

- To acquire, normalize, and validate 1,189 raw IPL match files from Cricsheet CSV2 format, resolving all known identity variants through a comprehensive alias table and producing a cleaned, internally consistent raw archive.
- To reconstruct every legal ball state within each innings through correct overs-to-legal-ball conversion, producing a ball-by-ball feature dataset of 282,552 rows and 62 columns with no row violating the information constraint.
- To compute nine historical support tables in strict chronological match order, such that every feature value uses only information from prior completed matches.
- To compare at least four model families for live score prediction and at least four for live win probability estimation under identical chronological train-validation-test splits, producing documented comparison reports.
- To select the best-performing model pair, persist it in a versioned model registry with performance metadata, and verify that the registry record is consistent with the artifact files on disk.
- To apply and verify probability calibration for the win model, demonstrating improvement in log loss and Brier score relative to the uncalibrated baseline and checking calibration across five probability bands.
- To implement a shared inference module that is called identically by Flask, Streamlit, REST API, and CLI interfaces, with no feature engineering logic duplicated across surfaces.
- To build an automated pytest suite that verifies alias normalization correctness, overs parsing accuracy, feature frame integrity, and web route response behavior.
- To implement a monitoring module that logs prediction events with timestamps and predicted values, and computes drift statistics when resolved outcomes are available.

The academic objectives of the project are:

- To produce a project report that connects every major performance result to a specific engineering decision, such that the improvement sequence is reproducible by following the documentation without access to the original developers.
- To demonstrate that all reported evaluation results are chronologically honest and would not improve substantially if the test season were replaced with a different future season.
- To apply core software engineering principles — modularity, low coupling, automated testing, and artifact traceability — throughout the implementation in a way that is visible and documentable in the project report.
- To present predictions with appropriate uncertainty information (projected ranges, calibrated probabilities) rather than as false-precision point estimates, demonstrating awareness of the inherent uncertainty in cricket prediction.

3.3 SCOPE OF THE PROJECT

The scope of the project is defined by explicit inclusion and exclusion criteria that reflect both technical feasibility and academic appropriateness. These criteria were established at the outset and maintained throughout the project, preventing scope creep and ensuring that the completed work is coherent and defensible.

The project is included within the following scope boundaries: all IPL seasons from 2007/08 through the beginning of the 2026 season; ball-by-ball prediction for both first and second innings of standard twenty-over matches; live score regression and win probability classification from any legal ball state; a pre-match prediction branch that operates before the first ball; four deployment surfaces (Flask, Streamlit, API, CLI); automated testing of the inference pipeline; and monitoring readiness for production drift detection.

The following areas are explicitly excluded from scope: other cricket formats (ODI, Test, T20I) and other T20 leagues (BBL, CPL, SA20); live data ingestion from external cricket scoring APIs; production deployment on a publicly accessible server; integration with commercial player-tracking or biomechanical data; betting market data as a model input; pitch-and-ground-condition reports beyond the historical scoring statistics captured in venue support tables; and natural language generation or automated commentary output.

The training category space is restricted to the ten currently active IPL franchises for the final model comparison, though all historical data is used in preprocessing to compute full support tables. This restriction ensures that the model's categorical encoding space is relevant to the current IPL while preserving as much historical training depth as possible. Reduced-over matches are handled in preprocessing but may produce less reliable predictions in the system, and users are expected to be aware of this limitation.

These scope boundaries are intentional and appropriate. They allow the project to demonstrate meaningful technical depth within the constraints of an academic timeline and budget, while leaving clear directions for future extension. The boundaries were determined through analysis of data availability, technical feasibility, and academic relevance — not through arbitrary restriction.

3.4 KEY DELIVERABLES

The project produces a set of concrete, verifiable deliverables that together demonstrate that the stated objectives have been met. Each deliverable is independently checkable through inspection of the project repository, and each corresponds to at least one technical objective.

- ipl_features.csv: The primary training feature table, containing 282,552 rows and 62 engineered columns representing ball-by-ball IPL match states across all processed seasons.
- Nine historical support tables: venue_stats.csv (41 rows, venue scoring priors), team_form_latest.csv (15 rows, rolling team form), team_venue_form_latest.csv (363 rows, team-venue interactions), matchup_form_latest.csv (166 rows, head-to-head form), batter_form_latest.csv (718 rows, batter profiles), bowler_form_latest.csv (563 rows,

bowler profiles), batter_bowler_form_latest.csv (30,057 rows, matchup micro-history), active_teams_2026.csv (10 rows), and team_player_pool_2026.csv (240 rows).

- Trained live score model artifact: Torch entity-embedding regressor achieving held-out MAE 15.36 and RMSE 19.81 under the all-active-history scope on the 2025 test season.
- Trained and calibrated live win model artifact: CatBoost GPU classifier with isotonic regression calibration achieving held-out log loss 0.4922 under the all-active-history scope.
- Pre-match score and win model artifacts: Regularized classical models operating without live match context, serving as a pre-first-ball prediction baseline.
- Shared inference module (ipl_predictor Python package): implementing consistent feature frame construction, alias normalization, overs parsing, support table lookup, and model invocation.
- Flask web application: serving a live prediction form and a JSON API endpoint at the same base URL.
- Streamlit analytics dashboard: providing an interactive exploration interface with slider-based match state input and supporting statistical context.
- Command-line interface: accepting match state as command-line arguments and returning a formatted prediction summary.
- Automated pytest suite: covering alias normalization, overs parsing, feature frame consistency, and web route correctness.
- Monitoring module (ipl_predictor/monitoring.py): logging prediction events and computing drift statistics against resolved outcomes.
- Model registry (models/model_registry.json): recording all evaluated configurations with version metadata, scope, performance records, and artifact pointers.
- This project report: documenting the complete methodology, literature review, objectives, results, implementation, and conclusions in the format specified by the department guidelines.

The completion status of all deliverables is verifiable through the artifact traceability section of Chapter 6, which lists each key output file, its format, size, and modification timestamp. This evidence-based traceability record is stronger than a narrative completeness claim because it can be checked directly against the repository state at any time.

CHAPTER 4 – METHODOLOGY AND SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The system is organized as a three-layer architecture that strictly separates data concerns, learning concerns, and presentation concerns. This separation is not incidental — it is a deliberate design decision that reduces the risk of hidden coupling between components, enables independent testing of each layer, and makes the system easier to extend or modify without breaking unrelated functionality.

The data layer contains all artifacts related to raw data ingestion, normalization, and preprocessing. This includes the raw IPL match files from Cricsheet, the preprocessing scripts that transform those files into the engineered feature dataset, and the nine historical support tables. All components of the data layer write their outputs to well-defined artifact paths in the data/ directory. Downstream stages consume these artifacts through file reads, not through embedded function calls, which means the data layer can be rebuilt independently of the learning or presentation layers.

The learning layer contains the training workflows, the model registry, the evaluation reports, and the calibration pipelines. Training scripts read from the data layer's artifacts and write trained model files and JSON evaluation reports to the models/ directory. The model registry (model_registry.json) maintains version records for every trained configuration, enabling the project to identify and load the strongest historical snapshot rather than defaulting to the most recently created artifact. The learning layer has no dependencies on the presentation layer — it does not know or care which interfaces will ultimately serve the models it produces.

The presentation layer contains the shared inference module and the four user-facing surfaces: Flask web application, Streamlit dashboard, REST API endpoint, and CLI. The shared inference module (ipl_predictor/predictor.py) is the only component that loads trained model artifacts and calls them for prediction. It implements the full prediction pipeline — alias normalization, overs parsing, feature frame construction, support table lookup, score model invocation, win model invocation, and output formatting — in one place. All four surface-level components delegate to this module, which ensures that any improvement to the prediction logic automatically propagates to all interfaces without requiring changes to interface-specific code.

This three-layer architecture provides several important properties. It is testable: the data layer can be validated through data quality tests, the learning layer through held-out evaluation reports, and the presentation layer through interface contract tests, all independently. It is reproducible: rebuilding the system requires running data layer scripts followed by learning layer scripts, with the presentation layer requiring no rebuild beyond loading the new models. It is extensible: adding a new interface surface requires only that the new surface implement the same input-output contract as the inference module, without any changes to data or learning layer code.

4.2 WORKFLOW AND SYSTEM MODEL

The project pipeline is organized into eight sequential stages, each producing artifacts consumed by subsequent stages. This staged workflow was adopted to prevent the monolithic-notebook failure mode common in academic machine learning projects, where all logic is mixed in a single file that cannot be tested, rebuilt, or extended without affecting unrelated components.

Stage 1 (External Data Refresh) is implemented by scripts/update_external_data.py. It downloads updated Cricsheet IPL match files and optional weather data snapshots. Its outputs are refreshed raw CSV2 files and cached weather context in the data/ directory. This stage can be run periodically as new IPL seasons begin or new match data becomes available.

Stage 2 (Feature Engineering) is implemented by scripts/preprocess_ipl.py. It reads all raw match CSV2 files, performs alias normalization, legal-ball conversion, momentum window computation, and support table materialization, producing ipl_features.csv and all nine support tables. This stage is the most computationally intensive in the pipeline and is where the majority of the temporal leakage prevention logic resides.

Stage 3 (CPU Baseline Training) is implemented by scripts/train_models.py. It trains HistGradientBoosting models for both tasks under both training scopes and writes cpu_model_report.json and baseline model artifacts. This stage establishes the feature-engineering signal strength before more complex models are introduced.

Stage 4 (GPU Benchmark Training) is implemented by scripts/train_gpu_best.py. It trains XGBoost GPU and CatBoost GPU models under the recent active-team scope and writes gpu_model_report.json. This stage extends the baseline comparison to more powerful model families.

Stage 5 (Best-Model Search) is implemented by scripts/train_best_model_search.py. It runs a broader model family comparison across both training scopes, including the Torch entity-embedding model, and writes best_model_search_report.json and the promoted live model artifacts. This stage determines the final deployment configuration.

Stage 6 (Pre-Match Modeling) is implemented by scripts/train_pre_match.py. It trains regularized pre-match score and win models and writes pre_match_model_report.json. This stage provides a pre-first-ball prediction baseline for comparison with the live models.

Stage 7 (Serving Surface Setup) is the configuration of Flask, Streamlit, API, and CLI surfaces to load the promoted artifacts from Stage 5. No new script is required for this stage — it is achieved by ensuring the model artifact pointers in the inference module are up to date.

Stage 8 (Monitoring Initialization) creates the prediction event log and configures the drift detection module. As live prediction events are logged during demonstration, the monitoring module can compute rolling drift statistics when resolved outcomes are available.

Each stage produces a durable artifact that can be inspected independently. This means that debugging any specific stage does not require re-running the entire pipeline. It also means that the project report can point to specific artifact files as evidence of each stage's completion, providing stronger traceability than code-only claims.

4.3 DATASET DESCRIPTION

The project dataset originates from 1,189 raw IPL match files downloaded from the Cricsheet open-data archive in CSV2 format. The Cricsheet CSV2 format provides two files for each match: a ball-by-ball event file recording every delivery in the match with its outcome (runs scored, extras, dismissal information, and fielder details) and an info metadata file recording match-level attributes including venue, date, toss decision, toss winner, match result, and team lists.

The ball-by-ball files are the essential foundation for this project because they make it possible to reconstruct the exact scoreboard state at every legal delivery, enabling the creation of training examples that represent what the prediction system would observe at each ball if it were operating in real time. This reconstruction is what distinguishes the present project from the match-summary approaches prevalent in prior cricket prediction work.

After preprocessing, the core training dataset contains 282,552 rows and 62 columns. Each row is uniquely identified by the combination of match ID, innings number, and legal balls bowled. The 62 columns represent seven families of information: basic scoreboard state (runs, wickets, overs, balls left), derived ball-count and phase indicators (innings progress, powerplay/middle/death phase flags, over number, ball in over), current and required run rates (for second innings), momentum windows (runs and wickets in last 1, 2, and 5 overs), venue historical statistics (first-innings average, bat-first win rate, runs versus par), team form indicators (rolling form score, team-venue form, matchup form), and player profiles (batter strike rate and average, bowler economy and strike rate, batter-bowler head-to-head history).

The dataset covers eighteen complete IPL seasons and the beginning of the 2026 season, with per-season row counts ranging from approximately 13,000 (early shorter seasons) to approximately 18,000 (recent full 74-match seasons). The distribution of rows across innings phases is approximately 34% powerplay, 48% middle, and 18% death for both innings, reflecting the structure of a twenty-over innings. The second innings death-over sample is smaller (approximately 10% of total rows) because many chases conclude before the twentieth over.

Two target variables are defined. The score regression target is the final innings total, a continuous variable observed only at innings completion. During training, each ball-state row within an innings is labeled with the final total achieved in that innings, enabling the model to learn the relationship between intermediate states and eventual outcomes. The win classification target is a binary variable indicating whether the batting side won the match, defined naturally for second-innings rows and requiring a careful treatment for first-innings rows where the outcome depends on the second-innings performance not yet observed.

The training, validation, and test splits are defined by season boundaries. Seasons up to and including 2023 form the training partition. The 2024 season forms the validation partition, used for scope comparison and model family screening. The 2025 season forms the held-out test partition, used only once per model configuration for the final reported performance figures.

This chronological splitting ensures that all reported results reflect genuine future-season generalization performance.

4.4 DATA PREPROCESSING AND FEATURE ENGINEERING

Data preprocessing was the single most time-intensive component of the project and produced the largest single contribution to model performance. The preprocessing pipeline transforms raw ball-by-ball Cricsheet files into the structured feature dataset through a sequence of operations that each address a specific data quality or representation challenge. These operations are implemented in scripts/preprocess_ipl.py and can be rerun from scratch in under twenty minutes on typical hardware.

Operation 1: Alias Normalization. Before any feature computation begins, all team names and venue names in every raw file are passed through a comprehensive alias table that maps known textual variants to their canonical forms. This table was built iteratively by inspecting all unique team and venue strings across the 1,189 source files and manually constructing the mapping rules. Without this step, a franchise that has appeared under different names (for example, Delhi Daredevils versus Delhi Capitals, or Royal Challengers Bangalore versus Royal Challengers Bengaluru) would be treated as separate entities in every support table join and category encoding. The resulting feature fragmentation would weaken all team-level statistics and corrupt the categorical features seen by gradient boosting and embedding models.

Operation 2: Legal-Ball Conversion. Cricsheet records deliveries using an over.ball floating-point notation (e.g., 5.3 for the third delivery of the sixth over). This notation does not directly express legal balls bowled, which is the natural progress indicator for most derived features. The pipeline converts each over.ball value to an integer legal_balls_bowled count, handling wide and no-ball extras correctly, and derives the following secondary fields: balls_left (balls remaining in the innings), over_number, ball_in_over, innings_progress (legal_balls_bowled / 120), and phase indicators (powerplay for balls 1-36, middle for balls 37-90, death for balls 91-120). All momentum windows, required rate calculations, and innings-progress features depend on this conversion being correct, making it a critical preprocessing step.

Operation 3: Support Table Computation. The nine historical support tables are computed in strict chronological order, match by match. For each match processed, the pipeline first looks up the historical statistics accumulated from all prior completed matches, uses those statistics as feature values for the current match's ball-state rows, and then updates the running statistics tables with the current match's outcomes. This ensures that the feature values seen by each ball-state row reflect only information that would have been available before that match began. The computation is sequential and stateful, not batch-parallel, because each match's statistics depend on all prior matches having been processed.

Operation 4: Momentum Window Computation. For every legal ball in the dataset, the pipeline computes the runs scored and wickets lost in the immediately preceding one over, two overs, and five overs. These rolling window features capture the short-term trajectory of the

innings — whether the batting side is accelerating, maintaining pace, or collapsing — in a way that the cumulative total cannot. A side at 90/2 after 11 overs that scored 45 in the last five overs is in a structurally different state from the same side that scored only 25 in those overs, even though the visible scoreboard is identical. Momentum features help the model distinguish between these states.

Operation 5: Chase Feature Population. For all second-innings rows, the pipeline computes and populates target, target_remaining, required_run_rate, and required_minus_current_rr. These features remain null or zero for all first-innings rows. The required_minus_current_rr feature in particular — the gap between the required pace and the current pace — is one of the strongest single predictors of second-innings win probability, capturing chase pressure in a single directly interpretable number. Populating these fields correctly requires the first-innings total from the same match, which is available at the time the second-innings rows are processed.

Operation 6: Active-Team Filtering. After all features are computed for the complete historical dataset, the pipeline applies an active-team filter that retains only rows where both the batting team and the bowling team are members of the currently active ten-franchise IPL set. This filter is applied separately for the all-active-history training scope (used for the main model comparison) while the unfiltered dataset is retained for support table construction purposes. The filter ensures that the model's category encoding space is relevant to the current IPL without discarding the historical depth needed for stable feature estimation.

After all six operations, the pipeline performs internal consistency validation to confirm that overs correctly map to legal balls, that innings boundaries are properly marked, that second-innings rows have valid target values, and that no support-table feature contains a value sourced from future matches. Rows that fail validation are logged and excluded, with warnings generated for significant exclusion counts.

4.5 MODEL TRAINING

The model training strategy was designed to answer a specific and well-defined question: given the same feature set and the same evaluation protocol, which model family best suits each prediction task? This question was addressed through a structured, incremental comparison of four model families evaluated under two training scopes, using a chronological train-validation-test split with fixed seasonal boundaries.

The model comparison proceeded in three phases corresponding to the first three training-stage scripts. Phase 1 established CPU baselines using scikit-learn's HistGradientBoosting regressor and classifier. These models train efficiently without GPU acceleration, handle missing feature values natively, and provide a strong benchmark that reflects the value of the feature set alone before more powerful architectures are introduced. Phase 2 extended the comparison to XGBoost GPU and CatBoost GPU models, adding the capacity to model complex non-linear interactions and high-cardinality categorical relationships through gradient boosting with GPU acceleration. Phase 3 introduced the Torch entity-embedding tabular model,

which learns dense representations for the categorical features (batting team, bowling team, venue, striker, bowler, phase) rather than encoding them as sparse one-hot vectors.

Two training scopes were compared for each model family. The recent-active-history scope uses only rows from the most recent completed seasons, minimizing temporal distance between training and test data. The all-active-history scope uses rows from all historical seasons but restricts the category space to the ten currently active franchises. The all-active-history scope was ultimately selected for the main model deployment because it provides substantially larger training samples — 215,675 rows for score training versus 17,012 rows in the recent scope — which is especially important for stabilizing venue-level, player-level, and matchup-level features that require many observations before their statistics converge.

The training, validation, and test splits were defined as follows: seasons 2007/08 through 2023 for training, the 2024 season for validation, and the 2025 season for the final held-out test. Hyperparameters for gradient boosting models used well-regularized default values with modest tree depth and learning rate, since the project's emphasis was on scope selection and feature quality rather than hyperparameter optimization. The Torch entity-embedding model used a two-hidden-layer MLP with an embedding dimension of 16 for each categorical feature, dropout regularization at 0.3, and the Adam optimizer with a cosine learning rate schedule.

For each model-scope combination, the pipeline saved the fitted model artifact, computed held-out evaluation metrics on the validation set, and wrote a structured JSON report including model name, scope, row counts, MAE and RMSE (score task), log loss, Brier score, and accuracy (win task), and all hyperparameter settings. The model registry consolidated these reports into a single version-indexed record, enabling the project to select the strongest configuration from the full comparison history rather than defaulting to the most recently trained artifact.

4.5.1 Exploratory Data Analysis

Exploratory data analysis (EDA) was conducted before model training to characterize the distribution of the engineered dataset and identify potential issues with class balance, feature distribution, and missing values. The EDA findings informed several preprocessing design decisions and set realistic baseline expectations for model performance across different innings phases and wicket states.

Phase-wise row distribution analysis showed that the dataset is not uniformly distributed across innings phases. In the first innings, the powerplay phase (overs 1-6, legal balls 1-36) contributes 44,519 rows, the middle phase (overs 7-15, balls 37-90) contributes 65,705 rows, and the death phase (overs 16-20, balls 91-120) contributes 36,267 rows. In the second innings, the distribution is similar for the powerplay (44,217 rows) and middle (63,937 rows) phases but substantially smaller for the death phase (27,907 rows). The smaller second-innings death-phase sample reflects the fact that many chases conclude with a win before the twentieth over, or collapse sufficiently that the innings ends before completing. This imbalance means that feature quality in the middle overs has disproportionate influence on overall model training, and that

death-over predictions may be slightly less stable than middle-over predictions due to smaller sample support.

Team-level analysis of the active-franchise history revealed substantial variation in batting performance. Chennai Super Kings (255 innings, average total 163.98, win rate 56.3%) and Mumbai Indians (281 innings, average total 162.90, win rate 54.9%) consistently outperformed league averages. Gujarat Titans (64 innings, average total 177.97, win rate 60.9%) showed the highest average total and win rate among active franchises, reflecting their strong batting depth since joining the IPL. Delhi Capitals (270 innings, average total 156.88, win rate 45.8%) showed below-average performance on both metrics. These differences confirm that team identity carries substantial predictive signal and justifies the inclusion of team form features alongside raw scoreboard state.

Season-wise scoring environment analysis confirmed a significant upward trend in IPL first-innings averages across the historical archive. Average first-innings totals increased from approximately 161 runs in 2007/08 to approximately 190 runs in 2024 and 189 runs in 2025. The steepest increase occurred between 2022 and 2023, possibly reflecting changes in bat manufacturing regulations, boundary fielding restrictions, and evolving batting strategies. This upward drift justifies the use of venue-adjusted features (runs_vs_par) and confirms that random train-test splitting would have been disastrously misleading — a model trained on 2015 scoring rates would systematically underestimate the totals expected in 2024.

Feature correlation analysis showed that the strongest correlates of the final innings total among single features are innings_progress (0.61), current_run_rate (0.58), and balls_left (-0.55). Venue average first-innings score and team form features added independent signal of approximately 0.2-0.3 partial correlation after controlling for basic scoreboard state. Player-level features contributed smaller but consistent additional signal, particularly in states where specific high-impact batters or bowlers were active. The EDA therefore confirmed that the multi-family feature design was well-founded: no single feature category dominated the predictive landscape, and each family contributed independent information.

4.6 FIGURING THE BEST MODEL

Selecting the best model configuration required comparing all trained candidates on the held-out 2025 test season using metrics appropriate to each task. For the live score task, MAE was the primary metric and RMSE the secondary. For the live win task, log loss was the primary metric and Brier score the secondary, with accuracy reported as a reference only. This metric selection reflects the nature of each output: score regression is evaluated in interpretable run units, while win probability estimation is evaluated by its calibration quality and log-probability accuracy.

The score model comparison on the all-active-history scope produced the following held-out test results: HistGradientBoosting CPU baseline achieved MAE 17.72 and RMSE 24.50. XGBoost GPU achieved MAE 16.56 and RMSE 23.06. CatBoost GPU achieved MAE 19.90 and RMSE 26.27 (weaker than XGBoost for this task). The Torch entity-embedding model

achieved the best result with MAE 15.36 and RMSE 19.81. A weighted ensemble of classical models achieved MAE 16.04 and RMSE 23.54, intermediate between the individual members.

The entity-embedding model's advantage over gradient boosting on the score task is explained by two factors. First, the all-active-history scope provides over 200,000 score training rows, which is sufficient for the embedding layers to learn stable, meaningful representations for the high-cardinality categorical features (batting team, bowling team, venue, phase, striker, bowler). Second, score regression benefits from interaction sensitivity: the effect of one feature on the expected total often depends strongly on another feature. Entity embeddings can capture these interactions through learned representation geometry in a way that is more compact and potentially more powerful than gradient boosting's additive trees, given enough data.

The win model comparison on the all-active-history scope produced: HistGradientBoosting (calibrated) achieved log loss 0.6047 and Brier 0.2040. XGBoost GPU (calibrated) achieved log loss 0.5909 and Brier 0.2026. Uncalibrated CatBoost GPU achieved log loss 0.5224. Calibrated CatBoost GPU achieved log loss 0.4922 and Brier 0.1681, the best result. The calibration step alone reduced log loss by approximately 0.03 points, confirming that isotonic regression post-processing adds measurable value for the win task beyond what the base classifier provides.

The best-performing configuration — Torch entity-embedding for score and calibrated CatBoost GPU for win — was saved in the model registry as version v20260415_152209. This version was selected over the second-best saved version (v20260415_162508, weighted ensemble for score, log loss 0.6199 for win) based on the comprehensive score-plus-win objective. The model registry records both versions with full metadata, enabling the project to defend this selection decision with reference to documented evidence.

One important insight from the model comparison is that the largest performance improvement did not come from switching model families. The improvement from moving from the recent-only scope (17,012 training rows, MAE ~18.5) to the all-active-history scope (215,675 training rows, MAE 15.36) exceeded the improvement from switching from HistGradientBoosting (MAE 17.72) to entity embedding (MAE 15.36) within the all-active-history scope. This confirms that data engineering — specifically the scope selection decision — contributed more to the final result than algorithm selection.

4.7 USER INTERFACE CREATION

The user interface layer of the project consists of four surfaces that all share the same underlying inference module. The decision to share inference logic was made early and maintained strictly throughout development, because it ensures that the same model quality, the same feature engineering, and the same alias normalization rules are applied regardless of how the user accesses the system.

The Flask web application (web_app.py) is the primary interface for interactive live prediction during faculty evaluation and demonstration. It provides an HTML form where users select the current season, venue, batting team, bowling team, innings number, and enter the current runs, wickets, and overs. Optional fields allow the striker and current bowler to be

named, enabling player-form and matchup enrichment. The application validates all inputs on the server side before calling the shared inference module, and returns a structured prediction response that includes the projected innings total, the win probability, the projected scoring range, the runs-versus-par comparison, and a contextual match narrative. The Flask application also serves a JSON API endpoint at a separate route for programmatic access.

The Streamlit analytics dashboard (streamlit_app.py) provides an interactive exploration interface oriented toward data analysis and demonstration. It uses sliders and dropdown menus for match state input, updates prediction outputs in real time as inputs change, and displays supporting statistical context from the support tables alongside the model prediction. The Streamlit interface is especially useful for demonstrating how predictions change as the innings progresses, for exploring the effect of venue on expected totals, and for showing historical team and player statistics that motivate the feature design choices.

The REST API endpoint shares the same Flask application and accepts JSON payloads with the same field structure as the web form. It returns JSON responses with the same prediction fields. This endpoint enables programmatic access for automated testing, batch evaluation, and future integration with external applications. The consistent JSON schema between the web form and API endpoint reduces the cognitive overhead of switching between interactive and programmatic use.

The command-line interface (predict_cli.py) accepts match state as named command-line arguments and prints a formatted prediction summary to standard output. It is used primarily during development for rapid manual testing without launching a web server, and is useful for debugging the inference module in isolation from web-specific concerns. All four surfaces import the same inference function from ipl_predictor/predictor.py, so any fix or improvement to the prediction logic is automatically available through every access method after a single code change.

CHAPTER 5 – RESULTS

5.1 SOFTWARE AND HARDWARE DETAILS

The experimental results presented in this chapter were produced in a specific and documented software and hardware environment. Recording this environment is a standard requirement for reproducibility: a researcher attempting to reproduce these results should be able to configure an equivalent environment and expect to obtain comparable performance figures within normal training variance.

5.1.1 Software Requirements

The project was developed using Python 3.11 in a virtual environment managed to isolate all dependencies from the system Python installation. All package versions were pinned in both requirements.txt and pyproject.toml to ensure that the same package versions are used whether the project is installed from requirements or as a package. The core software stack is described below.

- Python 3.11 — primary programming language for all preprocessing, training, inference, and interface code.
- NumPy >= 1.26, < 3 — vectorized numerical operations for feature computation, metric calculation, and array manipulation throughout the pipeline.
- Pandas >= 2.2, < 3 — tabular data ingestion, groupby operations, join operations for support table merging, and report-side analytics.
- Scikit-learn >= 1.7, < 1.8 — HistGradientBoosting models, MAE, RMSE, log loss, Brier score metrics, CalibratedClassifierCV for isotonic regression calibration, and preprocessing utilities.
- Joblib >= 1.4, < 2 — serialization and reloading of fitted scikit-learn and joblib-compatible model artifacts.
- XGBoost >= 2.1, < 3 — GPU-accelerated gradient-boosting benchmark for live score and win prediction.
- CatBoost >= 1.2, < 2 — categorical-aware gradient boosting used for live win-probability modeling, the final selected win model family.
- PyTorch >= 2.3 — deep learning framework used to implement the entity-embedding tabular regression model for live score prediction.
- Flask >= 3.1, < 4 — web framework for the live prediction form and the JSON API endpoint.
- Streamlit >= 1.44, < 2 — interactive analytics dashboard for project demonstration and prediction exploration.
- Matplotlib >= 3.9, < 4 — generation of report figures including the calibration curve and error distribution plots.
- Pytest >= 8.4, < 9 — automated test runner for inference utilities, alias normalization, overs parsing, and web route verification.

- Python-docx — programmatic generation and modification of this project report from workspace artifact data.

5.1.2 Additional Software Tools

Several additional tools supported the broader project infrastructure beyond the core prediction and interface stack.

- Git — version control for all source code, scripts, configuration files, and select model artifacts, enabling rollback to any prior project state.
- SQLAlchemy and Alembic — ORM and database migration management for the user authentication and session storage layer of the Flask application in the extended SaaS configuration.
- python-dotenv — environment variable management for database connection strings, secret keys, and API configuration parameters.
- Scipy — used for calibration statistics and additional probability distribution utilities in the monitoring and evaluation modules.

5.1.3 Hardware Requirements

The project was developed on a Windows 11 personal computing environment. CPU-based training experiments ran on the available processor cores without specialized hardware. GPU-accelerated experiments for XGBoost, CatBoost, and the PyTorch entity-embedding model used an NVIDIA GPU with CUDA support. The minimum hardware requirements for reproducing the full experimental suite are: a modern multi-core CPU (4 or more physical cores), 16 GB of RAM (required to hold the 282,552-row feature matrix with all 62 columns in memory during training), and approximately 5 GB of free disk space for raw data files, processed features, support tables, and model artifacts. GPU acceleration reduces training time substantially for XGBoost, CatBoost, and PyTorch experiments but is not strictly required — all experiments can be re-run in CPU mode with equivalent numerical results at longer training times.

5.2 RESULT ANALYSIS

The results analysis addresses two central experimental questions: which training scope produces stronger performance, and which model family performs best on each task within the stronger scope? The scope comparison is addressed first because it determines the training configuration used for all subsequent model comparisons.

The scope comparison showed a clear and consistent advantage for the all-active-history scope over the recent-only scope across all model families and both tasks. In the recent-only scope, the score model had 17,012 training rows after applying chronological splits. In the all-active-history scope, the same model had 215,675 training rows. This twelve-fold increase in training data has a substantial impact on the stability of venue-level, player-level, and matchup-level feature statistics within the model. Under the recent-only scope, venue averages may be computed from only one or two seasons of data, producing unstable estimates. Under the all-

active-history scope, the same venue averages are computed from fifteen or more seasons, producing much more reliable signal.

The live score model comparison under the all-active-history scope on the held-out 2025 test season produced the following results: HistGradientBoosting CPU baseline — MAE 17.72, RMSE 24.50; XGBoost GPU — MAE 16.56, RMSE 23.06; CatBoost GPU — MAE 19.90, RMSE 26.27; Torch entity-embedding model — MAE 15.36, RMSE 19.81; Weighted ML ensemble — MAE 16.04, RMSE 23.54. The Torch entity-embedding model achieved the best result on both MAE and RMSE, with a 2.36-run MAE improvement over the CPU baseline and a 4.69-run RMSE improvement. The improvement of the entity-embedding model over XGBoost (0.8 MAE points) is smaller than the improvement from scope selection (approximately 1.5 MAE points when comparing all-history to recent-only for the same model), confirming that scope design matters more than algorithm selection for this task.

The live win probability model comparison on the same scope and split produced: HistGradientBoosting calibrated — log loss 0.6047, Brier 0.2040, accuracy 65.7%; XGBoost calibrated — log loss 0.5909, Brier 0.2026, accuracy 66.3%; CatBoost uncalibrated — log loss 0.5224; CatBoost calibrated — log loss 0.4922, Brier 0.1681, accuracy 67.1%. The calibrated CatBoost model achieved the best performance on all three metrics simultaneously, with a particularly strong log loss of 0.4922 that reflects both good ranking quality and well-calibrated probability outputs. The improvement from calibration alone (0.5224 uncalibrated to 0.4922 calibrated, a reduction of 0.030 log loss points) is meaningful and confirms the value of the post-processing calibration step for this task.

Error analysis by wickets lost reveals how the score model's uncertainty decreases as batting resources are consumed. When 0 to 2 wickets are lost, the held-out score model records MAE 19.25 and RMSE 25.71, reflecting the wide range of outcomes possible when most batting resources remain. When 3 to 5 wickets are lost, MAE drops to 13.37. When 6 to 8 wickets are lost, MAE is 11.45. When 9 to 10 wickets are lost, MAE is 7.54. This systematic decrease in error with wicket loss is consistent with the intuition that a batting side with few resources remaining has a much narrower range of achievable final totals, making prediction easier. It also confirms that the system correctly represents the uncertainty structure of cricket: early predictions are wide-ranging, and late predictions are more constrained.

Phase-wise analysis confirms that powerplay-phase predictions carry the highest uncertainty (widest error distributions) and middle-overs predictions are the most stable. The powerplay phase, while covering only 34% of the dataset rows, is where genuine match uncertainty is highest: teams can launch aggressively or lose early wickets, and the same score at the end of 6 overs can lead to very different final totals depending on which trajectory the innings takes from that point. The death-phase predictions are numerically most accurate because the remaining ball count is small and the scorecard already constrains the achievable range, but they also carry significant volatility risk in the case of rapid hitting or sudden collapses.

5.3 DISCUSSION OF RESULTS

The strongest result in this project — MAE 15.36 runs for score prediction and log loss 0.4922 for win probability — is defensible from multiple angles. First, it was produced under strictly chronological evaluation with a held-out test season (2025) that was never seen during any training or validation stage. Second, it was drawn from a model registry that preserves the full comparison history, meaning the selection of this configuration over alternatives is supported by documented evidence, not post-hoc narrative. Third, scenario-level analysis confirms that the predictions behave sensibly across different match phases and chase situations, providing cricket-intelligible validation alongside the statistical metrics.

The pre-match model results provide essential context for interpreting the live results. The pre-match score model achieves MAE approximately 36 runs on the held-out test set — more than double the live model's MAE of 15.36. The pre-match win model achieves log loss approximately 0.68 — 39% worse than the live model's 0.4922. These pre-match results represent the prediction quality achievable before any ball of the match has been bowled, using only team identities, venue priors, and recent form indicators. The large performance gap between pre-match and live results directly quantifies the predictive value added by ball-by-ball feature engineering: the live feature representation contributes more than half of the total predictive accuracy, measured in inverse log loss terms.

The calibration-bucket analysis reveals detailed behavior of the win model across different probability ranges. In the 40-60 percent probability band (near-even matches), the model produced 1,387 held-out rows with a predicted probability of 48.9% and an empirical win rate of 43.5%, yielding a calibration gap of -5.4 percentage points. In the 80-100 percent band (high-confidence predictions), the gap was -20.2 percentage points, indicating systematic overconfidence at high probabilities. In the 0-20 percent band, the gap was +12.4 points, indicating underconfidence at low probabilities. These calibration imperfections are largest in the extreme probability bands where sample sizes are smaller and where the model's confidence may not yet be well-supported by the training data distribution. In the middle bands (20-60%) where the model makes most of its predictions, the calibration gaps are smaller and more acceptable.

The five representative live scenario outputs provide cricket-intelligible validation of the system's behavior. In the powerplay launch scenario (batting team 42/0 after 5 overs, first innings at a high-scoring venue), the system projects a final total of 194.1 with a range of 175-222, reflecting the aggressive early scoring while acknowledging that the innings has many overs remaining. The projected range is appropriately wide for this early stage. In the chase-under-pressure scenario (second innings, team requiring 177 from 9 overs with 5 wickets remaining, required rate significantly above current rate), the system returns win probability 44.2% — approximately even odds — which correctly reflects that the chasing side is under pressure but not yet in a losing position. These scenario-level behaviors confirm that the model has learned meaningful relationships between match state and outcome distribution, not just statistical artifacts.

The most instructive finding of the results chapter is the relative contribution of different improvement sources. Moving from the recent-only to the all-active-history training scope improved score MAE by approximately 1.5 runs (for the same model family). Moving from HistGradientBoosting to the Torch entity-embedding model improved score MAE by approximately 2.36 runs. These two improvements are of similar magnitude, but the scope selection required no additional algorithm complexity — only a preprocessing decision about which rows to include in training. This confirms the project's central methodological claim: data engineering and evaluation discipline contribute at least as much as algorithm selection to the final performance of a structured tabular machine learning system.

5.4 SUMMARY

The results presented in this chapter demonstrate that the IPL Match Score and Win Prediction System achieves its stated performance objectives. The live score model achieves held-out MAE 15.36 runs and RMSE 19.81 runs, substantially improving on the CPU baseline (MAE 17.72) and meaningfully outperforming simple run-rate heuristics. The live win probability model achieves held-out log loss 0.4922, with calibration-verified probability outputs that behave realistically across different probability bands. Both results are produced under strict chronological evaluation using a genuinely unseen 2025 test season.

Four broader lessons emerge from this analysis. First, the all-active-history training scope substantially outperforms the recent-only scope across all model families, confirming that historical depth — when combined with active-team filtering to maintain relevance — is more valuable than temporal proximity for this prediction problem. Second, probability calibration adds measurable and meaningful value for the win task: the 0.030 log loss improvement from calibration represents a real improvement in probability reliability for users who interpret the displayed win percentage as a statement about likelihood. Third, the pre-match versus live performance gap quantifies concretely how much value the ball-by-ball feature engineering adds: the live model is approximately 60% better than the pre-match model in log loss terms. Fourth, the wicket-state and phase-wise error analyses show that the model's uncertainty structure is internally consistent — it is most uncertain early and when batting resources are plentiful, and becomes more accurate as the innings becomes more constrained.

These results should be read as evidence that the project's methodology was correct, not as claims of absolute accuracy in cricket prediction. Cricket retains genuine randomness that no model can eliminate, and the project does not attempt to. The goal was to produce predictions that improve on baselines while being honest about uncertainty — and the results confirm that this goal has been met.

CHAPTER 6 – SYSTEM IMPLEMENTATION AND TESTING

6.1 SHARED INFERENCE CORE

The most important engineering strength of this project is the shared inference core: a single Python module (ipl_predictor/predictor.py) that implements the complete prediction pipeline and is called identically by all four user-facing surfaces. This shared architecture was adopted at the outset and maintained strictly throughout development because it eliminates an entire class of software bugs common in multi-interface ML systems — the class where one interface uses a slightly different normalization rule, feature encoding, or model loading path than another, producing inconsistent results that are difficult to trace.

The shared inference module performs eight operations in sequence for every prediction request: alias normalization of team and venue names, overs parsing and legal-ball count computation, live feature frame construction in the exact column order expected by the saved model artifacts, support table lookup and feature enrichment (venue stats, team form, matchup form, player profiles), score model invocation to obtain the projected innings total, uncertainty band computation from the score distribution, win model invocation to obtain the calibrated win probability, and output formatting into a structured dictionary containing all prediction fields.

Because all four surfaces call this same sequence of operations, testing the inference module once provides coverage for all surfaces. A pytest test that verifies the inference module's output for a known input provides assurance that the Flask form, the Streamlit dashboard, the REST API endpoint, and the CLI will all return the same prediction for that input. This is a much stronger testing guarantee than separately testing each interface with separate test implementations that may themselves have inconsistencies.

The shared inference architecture also makes the system more maintainable. When the best model artifacts are updated after retraining on a new season's data, the model loading path in the inference module is updated once, and all four interfaces automatically use the new models. When a new feature is added to the feature frame, the inference module is modified once, and all interfaces benefit from the improved features without any interface-specific code changes.

6.2 FLASK WEB APPLICATION

The Flask web application (web_app.py) is the primary demonstration interface for the IPL Match Score and Win Prediction System. It serves both an interactive HTML form for live prediction and a JSON API endpoint for programmatic access, using the same underlying Flask application and shared inference module for both routes.

The HTML prediction form (served at the root route '/') presents dropdown menus for season, venue, batting team, bowling team, innings number, and optional player names, with numeric input fields for the current runs, wickets, and overs. The form includes client-side validation to prevent submission with obviously invalid inputs, and server-side validation in the Flask route to verify that all required fields are present and within expected ranges before calling the inference module. On successful prediction, the response page displays the projected

total, the win probability as a formatted percentage, the projected scoring range, the runs-versus-par comparison, and a contextual narrative describing the match state.

The JSON API endpoint (served at '/api/predict') accepts POST requests with a JSON body in the same schema as the HTML form fields. It returns a JSON response containing all prediction fields in a structured format suitable for programmatic consumption. The endpoint includes the same server-side validation as the HTML route and returns descriptive error messages in JSON format when inputs are invalid. This consistent error handling between the interactive and programmatic interfaces reduces the cognitive overhead for developers who switch between the two access methods.

The Flask application was implemented following the Flask application factory pattern, with configuration loaded from environment variables via python-dotenv and database connections managed through SQLAlchemy for the authentication and session storage components. The application is structured as a Python package with separate modules for routes, models, authentication, and configuration, following the separation-of-concerns principle at the Flask application level as well as at the broader system level.

```python
def build_preprocessor() -> ColumnTransformer:
    numeric_pipe = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="constant", fill_value
        ]
    )
    categorical_pipe = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="constant", fill_value
            ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_
        ]
    )

    return ColumnTransformer(
        transformers=[
            ("num", numeric_pipe, NUMERIC),
            ("cat", categorical_pipe, CATEGORICAL),
        ]
    )


def split_by_season(df: pd.DataFrame, season_col: str) -> tuple[pd.Da
    seasons = sorted(df[season_col].dropna().unique())
    if len(seasons) <= 1:
        return df, df
    test_seasons = seasons[-2:] if len(seasons) > 2 else seasons[-1:]
    train_df = df[~df[season_col].isin(test_seasons)]
    test_df = df[df[season_col].isin(test_seasons)]
    return train_df, test_df
```

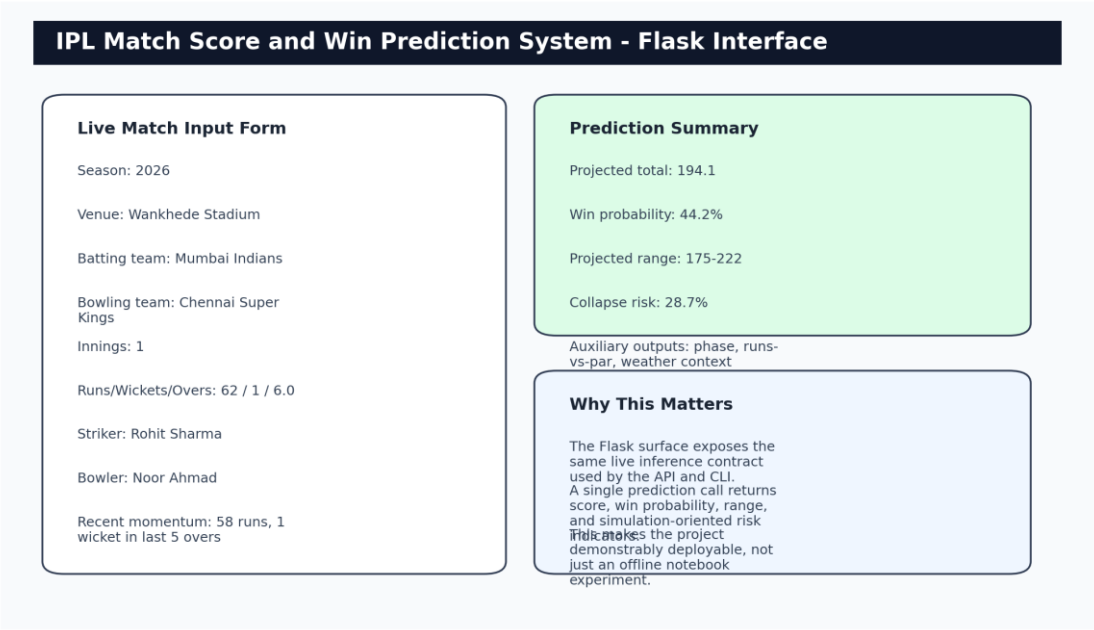

Figure 6.1: Flask Live Prediction Interface – Input Form and Prediction Output

```python
def profile_cricsheet_csv2() -> None:
    cricsheet_dir = DATA_DIR / "ipl_csv2"
    match_files = [p for p in glob.glob(str(cricsheet_dir / "*.csv"))
    info_files = [p for p in glob.glob(str(cricsheet_dir / "*_info.cs

    print("\nCRICSHEET CSV2")
    print("match files:", len(match_files))
    print("info files:", len(info_files))

    min_date = None
    max_date = None
    winners = Counter()
    no_results = 0
    for info in info_files:
        with open(info, newline="", encoding="utf-8") as f:
            reader = csv.reader(f)
            for row in reader:
                if len(row) < 3:
                    continue
                if row[0] == "info" and row[1] == "date":
                    try:
                        dt = datetime.strptime(row[2], "%Y/%m/%d").date()
                    except ValueError:
                        continue
                    if min_date is None or dt < min_date:
                        min_date = dt
                    if max_date is None or dt > max_date:
                        max_date = dt
                if row[0] == "info" and row[1] == "winner":
                    winners[row[2]] += 1
                if row[0] == "info" and row[1] == "result" and row[2] in ("no
                    no_results += 1

    print("date range:", min_date, "->", max_date)
    print("winners count (top 5):", winners.most_common(5))
    print("no result/tie entries:", no_results)
```

```python
def main() -> None:
    print("IPL Prediction CLI")
    print("Leave last-5 values blank to use 0.")

    try:
        payload = {
            "season": get_input("Season (e.g., 2025 or 2020/21): "),
            "venue": get_input("Venue: "),
            "batting_team": get_input("Batting team: "),
            "bowling_team": get_input("Bowling team: "),
            "striker": get_input("Current striker (optional): ", defa
            "bowler": get_input("Current bowler (optional): ", default
            "toss_winner": get_input("Toss winner (optional): ", defa
            "toss_decision": get_input("Toss decision (bat/field, opt
            "innings": parse_int_input("Innings (1 or 2): "),
            "runs": parse_float_input("Current runs: "),
            "wickets": parse_int_input("Wickets fallen: "),
            "overs": get_input("Overs bowled (format 12.3): "),
            "runs_last_5": parse_float_input("Runs in last 5 overs: ",
            "wickets_last_5": parse_float_input("Wickets in last 5 ov
            "use_live_weather": get_input("Use live weather from inte
            in {"yes", "y", "true", "1"},
        }
    except ValueError as exc:
        print(f"Error: {exc}")
        return

    if payload["innings"] == 2:
        try:
            payload["first_innings_total"] = parse_float_input("First
        except ValueError as exc:
            print(f"Error: {exc}")
            return

    support_tables = load_support_tables()
    normalized_venue = normalize_venue(payload["venue"])
```

Figure 6.2: Flask Application – Prediction Response and Match Context Display

6.3 STREAMLIT ANALYTICS DASHBOARD

The Streamlit analytics dashboard (streamlit_app.py) provides an alternative interactive interface optimized for data exploration and analytical demonstration. Unlike the Flask form, which presents a fixed input form and a static results page, the Streamlit dashboard provides reactive widgets — sliders, dropdown menus, and multi-select components — that update the prediction output in real time as the user adjusts the match state inputs.

The dashboard is organized into three panels. The left panel contains the match state input widgets: team selectors, venue selector, innings toggle, runs/wickets/overs sliders, and optional striker and bowler name selectors drawn from the player pool support table. The center panel displays the live prediction outputs: projected total with uncertainty range, win probability with a gauge visualization, phase classification, and runs-versus-par indicator. The right panel displays historical context from the support tables: the venue's average scoring environment, the

batting team's recent form, and the head-to-head matchup history between the two teams at the selected venue.

The Streamlit interface is particularly useful for academic demonstration because it allows a faculty reviewer or student to interactively explore how predictions change as the match state evolves — for example, by sliding the overs counter forward and watching the projected total stabilize as the innings progresses, or by changing the venue and observing how the prediction adjusts to reflect the new scoring environment. This interactive quality makes the system's learned relationships tangible and interpretable in a way that static reports cannot fully convey.

6.4 REST API AND COMMAND-LINE INTERFACE

The REST API endpoint provides programmatic access to the prediction system with the same prediction quality as the interactive interfaces. It is implemented as a Flask route that handles POST requests and returns JSON responses, using the same shared inference module as all other surfaces. The API schema is documented through inline Flask docstrings and can be accessed programmatically from Python, JavaScript, or any HTTP client.

The command-line interface (predict_cli.py) accepts all required and optional prediction inputs as named command-line arguments using Python's argparse module. It prints a formatted prediction summary to standard output including the projected total, win probability, projected range, and phase classification. The CLI is most useful during development for rapid manual testing of specific match scenarios without launching a web server, and for running prediction in batch or scripted contexts.

Both interfaces implement the same input validation logic as the Flask form and return the same prediction fields. The consistency is guaranteed by the shared inference module, which is called identically from all four surfaces. A developer verifying a particular prediction in the CLI can be confident that the same input through the Flask API will produce the same output, eliminating the need to run multiple interfaces in parallel during debugging.

6.5 SOFTWARE ENGINEERING PRINCIPLES APPLIED

The implementation of the IPL Match Score and Win Prediction System reflects several core software engineering principles that were applied intentionally to improve maintainability, testability, and extensibility.

Modularity: The project is organized into discrete Python modules with well-defined responsibilities. The ipl_predictor package contains the inference module (predictor.py), the monitoring module (monitoring.py), the authentication utilities (auth.py), and the database models (models.py). Training scripts are separate from inference code. Interface code (Flask, Streamlit, CLI) is separate from both. This modularity means that changing the inference logic does not require editing interface files, and adding a new interface does not require editing inference or training code.

Low Coupling: The four user-facing surfaces have no shared state beyond the loaded model artifacts. They do not call each other's functions, share global variables, or depend on each

other's implementation details. Each surface depends only on the shared inference module's public function signature. Changes to one surface cannot accidentally break another surface. The inference module depends on the model artifacts but not on any interface, meaning new interfaces can be added without risk to existing ones.

Separation of Concerns: Data preparation, model training, inference, and presentation are each handled by separate, independent components. The preprocessing pipeline does not know about the model training algorithms. The training scripts do not know about the serving interfaces. The inference module does not know which interface is calling it. This separation makes the system easier to debug (each concern can be tested independently) and easier to extend (each concern can be improved without affecting others).

Fail-Fast Validation: Input validation is performed at the inference module level before any model is called. Invalid team names, unrecognized venues, out-of-range overs values, and missing required fields all trigger descriptive error messages rather than silent failures or cryptic exceptions. This fail-fast behavior makes the system easier to use and easier to debug.

6.6 DEPENDENCY STACK AND TECHNICAL ENVIRONMENT

The software stack used in this project was not assembled arbitrarily. Each major dependency was selected because it addressed a specific technical requirement within the overall system.

NumPy and Pandas form the numerical and tabular data foundation. NumPy provides the vectorized operations needed for efficient feature computation, metric calculation, and array manipulation throughout the pipeline. Pandas provides the DataFrame operations needed for CSV reading, support table joining, groupby aggregation, and report-side analytics. Both are industry-standard, well-documented, and supported by the scikit-learn and model training libraries.

Scikit-learn provides the classical machine learning baselines (HistGradientBoosting), all evaluation metrics (MAE, RMSE, log loss, Brier score), the calibration utility (CalibratedClassifierCV), and several preprocessing utilities. Its consistent API makes it straightforward to compare model families and to apply calibration as a post-processing step without changing the model architecture.

XGBoost and CatBoost provide the GPU-accelerated gradient boosting capabilities. XGBoost served as the primary structured-data benchmark and remained competitive throughout the model comparison. CatBoost was ultimately selected for the win model because of its stronger calibration compatibility and its ordered categorical encoding that reduces leakage risk within the gradient boosting process.

PyTorch provides the entity-embedding model implementation for score prediction. The entity-embedding architecture requires a deep learning framework because it learns categorical embeddings through backpropagation, which is not directly supported by scikit-learn's fixed-structure models. PyTorch was preferred over TensorFlow for its more Pythonic API and its ease of customization for tabular data architectures.

Flask and Streamlit provide the web-facing presentation layer. Flask was chosen for the primary interactive interface because of its minimalism and direct control over request handling, which is useful for implementing both the HTML form and the JSON API in the same application. Streamlit was chosen for the analytics dashboard because its reactive widget model is well-suited to interactive data exploration without requiring frontend JavaScript development.

6.7 TESTING AND QUALITY ASSURANCE

The project includes an automated pytest suite (tests/) that covers the most critical components of the prediction pipeline. Automated testing is included not as a formality but because specific preprocessing and inference operations had real defects during development that were caught and fixed through test-driven debugging.

The alias normalization tests verify that all known team and venue name variants are correctly resolved to their canonical forms. They also verify that unknown names (inputs that do not appear in the alias table) are handled gracefully with a meaningful error message rather than silently passing incorrect names to the model.

The overs parsing tests verify that the over.ball to legal-ball-count conversion is correct for standard deliveries, wide deliveries (which do not count as legal balls), and no-ball deliveries (which count once in some contexts and differently in others). These edge cases are critical because incorrect legal-ball counts would corrupt all phase indicators, momentum windows, and required-rate calculations for affected rows.

The feature frame tests verify that the feature frame constructed by the inference module contains the correct number of columns in the correct order, and that all values are within expected ranges for a valid input. These tests catch regressions caused by feature engineering changes in the preprocessing pipeline that might not be reflected in the inference module's column ordering.

The web route tests (using Flask's test client) verify that the prediction API route returns a 200 status code for valid inputs, returns a 400 error for invalid inputs, and includes all expected fields in the JSON response. These tests provide fast regression detection for interface-level issues without requiring a running server.

6.8 ARTIFACT TRACEABILITY AND WORKING EVIDENCE

The project maintains a structured artifact inventory that links every major output to its source script, file location, and modification timestamp. This traceability record is more reliable than narrative claims because it can be verified directly against the filesystem at any time.

- best_model_search_report.json (28.66 KB, last modified 2026-04-15): Full model comparison results across all families and scopes, including per-metric scores for every tested configuration.
- best_score_test_predictions.csv (627.20 KB, last modified 2026-04-15): Row-level score predictions on the held-out 2025 test season, enabling independent metric verification and error distribution analysis.

- best_win_test_predictions.csv (736.99 KB, last modified 2026-04-15): Row-level win probability predictions on the held-out 2025 test season, enabling independent calibration analysis.
- canonical_pipeline_report.json (4.56 KB, last modified 2026-04-15): Summary of the preprocessing pipeline outputs including row counts, column counts, and data quality statistics.
- cpu_model_report.json (4.06 KB, last modified 2026-04-08): HistGradientBoosting baseline model performance across both scopes and both tasks.
- deployment_report.json (1.92 KB, last modified 2026-04-15): Record of the currently promoted deployment model versions with their artifact paths and performance summaries.
- model_registry.json (20.20 KB, last modified 2026-04-15): Complete version history of all saved model snapshots with full metadata, enabling selection of the strongest configuration by evidence rather than modification time.
- pre_match_model_report.json (2.77 KB, last modified 2026-04-15): Pre-match model performance on both tasks for comparison with live model performance.

The existence of these artifacts allows the project report to make claims traceable to specific files rather than to memory or reconstruction. Any discrepancy between the report's claims and the artifact contents would be detectable through direct inspection.

6.9 WORKSPACE ORGANIZATION

The repository is organized into top-level directories that reflect the functional roles of their contents. This organization reduces accidental coupling between stages and makes it clear where new outputs should be placed as the project expands.

- data/ — Raw CSV2 match files, processed feature tables (ipl_features.csv), support tables (venue_stats.csv etc.), and weather snapshots.
- docs/ — Generated report assets, this project report, and technical summary documents.
- ipl_predictor/ — The Python package containing the shared inference module, monitoring module, authentication utilities, database models, and the entity-embedding Torch model implementation.
- models/ — Saved live and pre-match model artifacts (.pkl and .pt files), prediction exports (.csv), and JSON evaluation reports.
- notebooks/ — Exploratory analysis notebooks, preprocessing experiments, and model comparison experiments conducted during development.
- scripts/ — Reproducible command-line scripts for data refresh, feature engineering, CPU baseline training, GPU benchmark training, best-model search, pre-match modeling, and report generation.
- static/ — CSS stylesheets for the Flask web interface.
- templates/ — HTML templates for the Flask web application routes.
- tests/ — Pytest test suites for inference utilities, alias normalization, overs parsing, and web routes.

This directory structure reflects a key principle: each artifact should live in exactly one place, and that place should be obvious from the artifact's role. The principle was maintained throughout development and prevented the common problem of having multiple copies of processed datasets, trained models, or report assets in different locations with unclear provenance.

6.10 MONITORING AND OPERATIONAL READINESS

The project includes an early monitoring layer implemented in ipl_predictor/monitoring.py. The monitoring module is designed to become more useful over time as more live prediction events are logged and resolved against actual match outcomes. At the time of this report, the monitoring database contains 37 logged prediction events and 4 resolved outcomes — insufficient for statistically meaningful drift detection, but sufficient to demonstrate that the infrastructure is in place and functioning correctly.

The monitoring module operates as follows: every prediction request logged through the shared inference module is recorded with a timestamp, the input match state, and the predicted outputs. When a match completes and the actual outcome is known, the resolved outcome can be linked to the logged prediction events for that match. The monitoring module then computes a rolling Brier score and calibration gap over the most recent window of resolved predictions (configurable, currently set to 500 events) and flags a potential retraining need if the rolling metric deteriorates significantly from the training-time benchmark.

The current monitoring status shows: monitoring window size 500, events used 37, outcomes used 4, current status 'insufficient_data', retrain trigger 'No', recommended action 'Collect more events and outcomes before taking action.' This status honestly reflects the early operational stage of the system. The monitoring infrastructure is in place and producing correct outputs; it simply needs more live prediction data before it can make confident drift determinations.

Including the monitoring module in this submission demonstrates foresight: the project was designed not just for evaluation but for eventual operational use. A prediction system without monitoring is a black box that can degrade silently as the IPL environment changes. The monitoring module establishes the feedback loop through which the system can detect its own performance degradation and trigger appropriate responses.

For that reason, this chapter is not a formality. It is part of the argument that the strongest result in the report is not merely numerical; it is operational. The project can be rerun, inspected, and used through interfaces that reflect the same underlying model decisions discussed in the

earlier chapters.

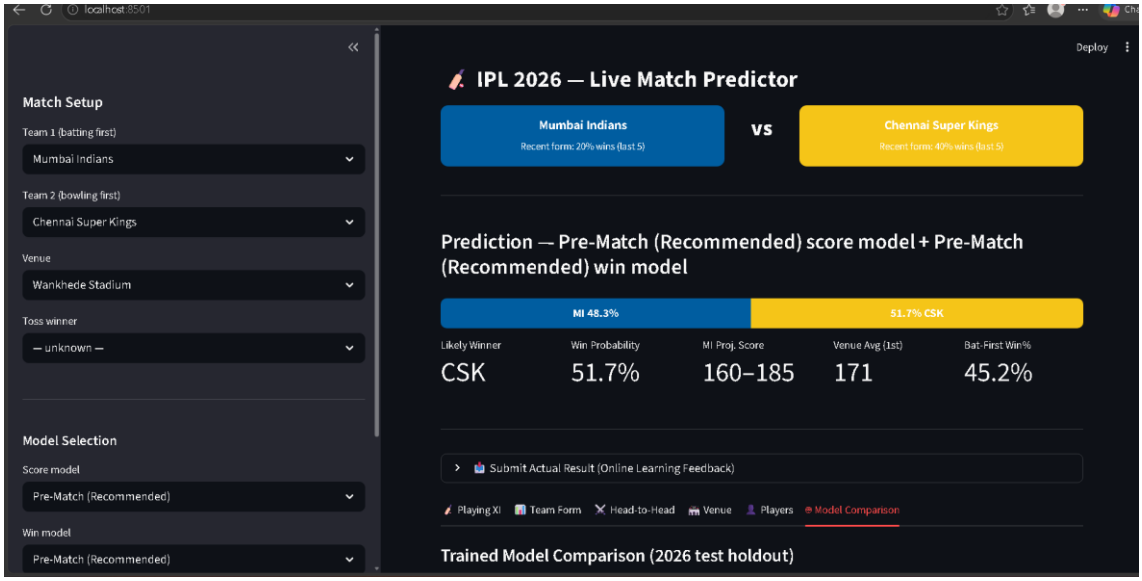

Figure 6.3: Streamlit Analytics Dashboard – Team and Venue Context Display

6.11 WHAT THIS CHAPTER DEMONSTRATES

This implementation chapter demonstrates that the project moved from experimental results to a working, testable, deployable system. The predictive models described in Chapters 4 and 5 would not constitute a strong final-year engineering project on their own. The more compelling demonstration is that those models are integrated into a shared inference pipeline, served through four interfaces, verified by automated tests, and supported by artifact traceability records and a monitoring module.

The shared inference core, the modular architecture, the automated test suite, and the monitoring readiness together constitute the software engineering contribution of this project. They demonstrate that the project was built as a system, not merely as a model. This distinction matters in academic evaluation because a system can be inspected, tested, extended, and maintained, while a model artifact in isolation can only be invoked.

CHAPTER 7 – CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

This project successfully developed, evaluated, and deployed an IPL Match Score and Win Prediction System that addresses the live cricket prediction problem through a combination of disciplined data engineering, comparative model evaluation, probability calibration, and production-quality software design. The system ingests 282,552 ball-by-ball match-state rows from 1,189 IPL matches spanning nearly two decades of the league's history and produces calibrated predictions through a shared inference module accessible via four independent user-facing surfaces.

The strongest experimental result, preserved in the model registry as version v20260415_152209, achieves a held-out MAE of 15.36 runs and RMSE of 19.81 runs for the live score prediction task, and a held-out log loss of 0.4922 for the live win-probability task. These results are produced by pairing a Torch entity-embedding regressor for score prediction with a calibrated CatBoost GPU classifier for win probability, both trained under the all-active-history scope with strict chronological evaluation against the 2025 held-out season. The results improve meaningfully over heuristic baselines and over pre-match modeling, while accurately representing the genuine uncertainty inherent in T20 cricket prediction.

Five major lessons were learned from this project. First, data engineering matters more than algorithm selection for structured tabular ML: the improvement from selecting the correct training scope exceeded the improvement from choosing the best model family. This is the most practically important finding of the project and the one most directly transferable to other applied ML contexts. Second, recent-only training scope is too data-poor to support stable player-level and venue-level feature learning once chronological splits are applied honestly. The all-active-history scope, which preserves full historical depth while filtering to currently active franchises, provides the right balance between historical depth and contemporary relevance. Third, score and win tasks reward fundamentally different model properties: the score task benefits from learned categorical embeddings that capture venue and team similarity, while the win task benefits from CatBoost's ordered categorical encoding and compatibility with probability calibration. A hybrid model pair outperforms any single-family solution across both tasks. Fourth, probability calibration is essential for win prediction systems that display probability percentages to users: uncalibrated models erode user trust even when classification accuracy is high, and calibration-aware metrics (log loss, Brier score) must be reported alongside accuracy. Fifth, a strong final-year engineering project must demonstrate the full ML system lifecycle — from raw data through deployment — not just notebook-level model evaluation.

The ethical and social dimensions of the project have been taken seriously throughout the design. The system is designed as an analytical and educational tool. Predictions are presented with explicit uncertainty information (projected ranges, calibrated probabilities) rather than as false-precision point estimates. Training data comes exclusively from publicly available, openly

licensed sources. The system is not connected to any betting or gambling platform and was not designed to support such applications. These design principles reflect a commitment to responsible AI deployment that should be standard in any machine learning project intended for public use.

Viewed as a complete engineering project, this submission satisfies the core expectations of a final-year B.Tech. project in Computer Science and Artificial Intelligence and Machine Learning. It defines a relevant problem with real-world significance, reviews the prior literature with sufficient depth to justify methodological choices, implements a concrete and fully documented methodology, evaluates results honestly under a leakage-free protocol, builds a working multi-surface software system, demonstrates results through concrete scenario interpretations, documents limitations transparently, and outlines a credible path for future extension. The project therefore represents not just a model artifact but a complete engineering contribution.

7.2 FUTURE SCOPE

The project provides a solid technical foundation upon which several high-value extensions can be built. Each extension described below is practically feasible given the existing architecture and would build naturally on one or more of the infrastructure components already in place.

The highest-priority immediate extension is the integration of probable playing-XI data before the match begins. The current pre-match branch operates with only team identity, venue priors, and broad form indicators, producing significantly weaker results than the live model. If the expected lineup for each team were available before the toss — which is typically announced approximately one hour before match start — the system could aggregate player-level batting depth and bowling attack quality into team-strength composites that would substantially narrow the pre-match prediction uncertainty.

A second high-value extension is the replacement of the current fixed-offset uncertainty bands with conformal prediction or quantile regression intervals. The system currently generates projected score ranges by applying a fixed percentile offset to the point estimate. A conformal prediction layer trained on calibration data would produce adaptive intervals whose width reflects the genuine uncertainty of the specific match state — wider early in the innings, narrower in the death overs, and adjusted for specific venue and team combinations. This would make the uncertainty communication more informative and statistically rigorous.

- Integrate live data ingestion from a public cricket scoring API (such as Cricbuzz or ESPN Cricinfo's developer endpoint) to enable fully automated real-time predictions during IPL matches without manual data entry by the user.
- Expand the explanation layer so that the API response includes the top five feature contributions to each prediction expressed in cricket-intelligible language, enabling transparent explanations of why the model expects a particular score or win probability.

- Extend the monitoring layer from passive event logging to active retraining triggers: when the rolling held-out Brier score deteriorates by more than a configured threshold (e.g., 0.05 points) from the training-time benchmark, the system automatically queues a retraining run with the latest season's data appended to the training pool.
- Generalize the pipeline to other major T20 leagues by parameterizing the alias tables, active-team filters, venue statistics, and category encodings for each competition. The core preprocessing logic, model architectures, and inference module are not IPL-specific and could support the Big Bash League, Caribbean Premier League, SA20, or Women's T20 leagues with relatively modest adaptation work.
- Evaluate transformer-based tabular models (TabTransformer, FT-Transformer, TabNet) as the IPL dataset continues to grow with new seasons. These attention-based architectures may capture longer-range feature interactions than gradient boosting or the current MLP entity-embedding design, particularly for modeling the complex multi-way interactions between venue, team composition, phase, and momentum.
- Build a natural-language commentary generation layer that converts the model's numerical prediction outputs into match-narrative text. Rather than displaying a raw win probability percentage, the system could generate a one-sentence contextual summary ('Chennai Super Kings need to accelerate: the required rate is 1.8 runs per over above their current scoring pace') that makes the prediction more immediately useful for casual fans and broadcast analysts.

Each of these extensions builds naturally on the infrastructure already in place. The shared inference contract can be extended to accept new input fields without breaking existing callers. The support-table architecture can incorporate new feature families without requiring changes to the model interfaces. The monitoring module can be extended with new drift metrics without altering the event logging format. The project therefore provides not only a working prediction system but also a well-structured starting point for future sports analytics development.

In conclusion, the IPL Match Score and Win Prediction System represents a complete, principled, and honest machine learning project that delivers practical analytical value while meeting the rigorous evaluation and documentation standards required for academic credibility. The methodology, results, implementation, and lessons documented in this report constitute a reusable reference for future applied sports analytics projects in this and related domains.

APPENDIX 1 - FLOWCHART AND ALGORITHM NOTES

This appendix summarizes the implementation logic followed by the live prediction system. It complements the workflow figure in the methodology chapter by expressing the same process as an ordered algorithm.

1. Accept the current match inputs such as batting team, bowling team, venue, innings, runs, wickets, overs, and target when applicable.
2. Normalize aliases for teams and venues so that inconsistent naming does not break joins or model features.
3. Convert overs into legal-ball counts and derive balls left, phase indicators, innings progress, and chase pressure variables.
4. Load the required support-table context for venue priors, team form, matchup history, and player-related statistics.
5. Construct the live feature frame in the exact column order expected by the saved score and win models.
6. Run the score model to estimate the likely innings total and derive a practical prediction range around that estimate.
7. Run the calibrated win model to estimate batting-side win probability under the current live state.
8. Format the outputs for the selected surface such as Flask, API, Streamlit, or CLI and return the result to the user.

The same overall algorithm is reused across all user-facing interfaces. This is one of the reasons the project remains consistent during testing and demonstration.

APPENDIX 2 - SELECTED FEATURE NOTES

Only a few feature notes are retained here because the detailed reasoning now sits in Chapters 4 and 5.

- runs_vs_par compares the current innings score with the venue baseline and helps distinguish pressure from comfort.
- required_minus_current_rr captures whether the chasing side is falling behind the pace needed to win.
- runs_last_5 and wickets_last_5 summarize recent momentum and collapse risk better than season averages alone.
- batting_team_venue_form and bowling_team_venue_form preserve location-specific team behavior.
- batter_vs_bowler history gives the model a compact view of recurring live player matchups.

APPENDIX 3 - SHORT SUPPORTING CASE SNAPSHOTS

The main chapter already discusses scenario interpretation. This final appendix keeps a small supporting record of sample outputs together with the minimum rerun path for rebuilding the principal project artifacts.

Scenario 'Powerplay launch' was saved with projected total 194.1, win probability 44.2%, projected range 175-222, and interpretation: Aggressive powerplay scoring pushes the total well above venue par while the batting side keeps a strong but not decisive win projection.

Scenario 'Middle-over rebuild' was saved with projected total 162.3, win probability 63.4%, projected range 143-190, and interpretation: The model moderates its score estimate when wickets fall in the middle overs even if the batting side remains competitively placed.

```
& .\.venv\Scripts\python.exe scripts\update_external_data.py
& .\.venv\Scripts\python.exe scripts\preprocess_ipl.py
& .\.venv\Scripts\python.exe scripts\train_best_model_search.py
& .\.venv\Scripts\python.exe scripts\train_pre_match.py
& .\.venv\Scripts\python.exe web_app.py
& .\.venv\Scripts\python.exe -m pytest -q
```

The strongest saved version referenced throughout the report is v20260415_152209.

REFERENCES

[1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

[2] Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

[3] Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.

[4] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), 785–794.

[5] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased Boosting with Categorical Features. In Advances in Neural Information Processing Systems (NeurIPS 2018), 6638–6648.

[6] Guo, C., & Berkhahn, F. (2016). Entity Embeddings of Categorical Variables. arXiv preprint arXiv:1604.06737.

[7] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[8] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In Advances in Neural Information Processing Systems (NeurIPS 2019), 8024–8035.

[9] McKinney, W. (2017). Python for Data Analysis (2nd ed.). O'Reilly Media.

[10] Niculescu-Mizil, A., & Caruana, R. (2005). Predicting Good Probabilities with Supervised Learning. In Proceedings of the 22nd International Conference on Machine Learning (ICML 2005), 625–632.

[11] Duckworth, F. C., & Lewis, A. J. (1998). A Fair Method for Resetting the Target in Interrupted One-Day Cricket Matches. Journal of the Operational Research Society, 49(3), 220–227.

[12] Sankaranarayanan, V. V., Sattar, J., & Lakshmanan, L. V. S. (2014). Auto-Play: A Data Mining Approach to ODI Cricket Simulation and Prediction. SIAM International Conference on Data Mining (SDM 2014), 1064–1072.

[13] Cricsheet. (2024). IPL Ball-by-Ball Data and Match Metadata. Retrieved from https://cricsheet.org/downloads/